

Tightly Secure Threshold Signatures over Pairing-Free Groups

Renas Bacho^{1,2} 

Benedikt Wagner³ 

January 9, 2026

¹ CISP Helmholz Center for Information Security

renas.bacho@cispa.de

² Saarland University, Saarbrücken, Germany

³ Ethereum Foundation

benedikt.wagner@ethereum.org

Abstract

Threshold signatures have been drawing lots of attention in recent years. Of particular interest are threshold signatures that are proven secure under adaptive corruptions (NIST Call 2023). Sadly, existing constructions with provable adaptive security suffer from at least one of the following drawbacks: (i) strong idealizations such as the algebraic group model (AGM), (ii) an unnatural restriction on the corruption threshold being $t/2$ where t is the signing threshold, or (iii) prohibitively large security loss under established assumptions. Notably, point (iii) has received little to no attention in the literature on this subject.

In this work, we introduce Twinkle-T, a new threshold signature scheme which overcomes these limitations. Twinkle-T is the first scheme to have a fully tight security proof under up to t adaptive corruptions without relying on the AGM. It also has a signing protocol consisting of only three rounds and thus matches the currently best threshold signature with full adaptive security Twinkle (Eurocrypt 2024) in the pairing-free discrete logarithm setting. We prove security from a standard non-interactive assumption, namely, the Decisional Diffie-Hellman (DDH) assumption, in the random oracle model.

Keywords: Threshold Signatures, Tightness, Adaptive Security, Pairing-Free Groups

Contents

1	Introduction	3
1.1	Our Contribution	4
1.2	More on Related Work	5
1.3	Paper Organization	6
2	Technical Overview	6
2.1	Twinkle and its Security Proof	6
2.2	Towards Tight Threshold Partitioning	7
2.3	Reducing the Number of Rounds	8
3	Preliminaries	10
3.1	Threshold Signatures	10
3.2	Tagged Linear Function Families	13
4	Non-Interactive Arguments for Tagged Linear Functions	14
5	Our Construction	15
5.1	Construction	16
5.2	Security Analysis	17
6	Instantiations and Efficiency	24
A	More Details on Non-Interactive Argument Systems	34
A.1	Formal Definitions	34
A.2	Proofs for Our Construction	37
B	More Details on Our Instantiations	40
C	Pseudocode	41

1 Introduction

Multi-party variants of digital signatures have recently garnered a lot of attention due to their ability to distribute trust across multiple parties, a feature that proves beneficial in a range of modern applications, including cryptocurrencies. Arguably, one of the most prominent examples of multi-party signatures are *threshold signatures* [Des88, DF90, Ped91]. In a $(t + 1)$ -out-of- n threshold signature scheme, the secret key sk for a public key pk is secret-shared among n signers, ensuring correctness and security:

- *Correctness.* Any subset of $t + 1$ signers can collaborate to execute a signing protocol that produces a valid signature σ for a message m . This signature can then be verified against the public key pk and the message m .
- *Security.* No malicious coalition of t or fewer signers can generate a valid signature. This has to hold even if some of them have maliciously participated in signing interactions with honest parties before.

The growing interest in threshold signatures – and threshold cryptography more broadly – has even resulted in ongoing efforts towards standardization [BP22].

Progress in Pairing-Free Groups. Motivated by the appealing features of pairing-free cyclic groups [TZ22, TZ23], recent research has focused on constructing threshold signature schemes in such groups [KG20, BCK⁺22, TZ23, CKM23b, Lin24, BLT⁺24, KRT24]. In our work, we also focus on pairing-free constructions, in the random oracle model (ROM). Notably, in this setting, there has been significant progress in terms of security guarantees:

- *Non-Interactive Assumptions.* While early constructions had to rely on interactive assumptions for their security proof, Tessaro and Zhu [TZ23] pioneered a shift towards more conservative, non-interactive assumptions.
- *Adaptive Security.* Early schemes are analyzed only under static corruptions, where the adversary must corrupt parties before learning the public key. More recent developments have tackled the stronger, more realistic scenario where the adversary can adaptively corrupt parties at any time. Namely, Crites et al. [CKM23b] showed a restricted form of adaptive security for their scheme, where the adversary is only allowed to corrupt $t/2$ parties, under an interactive assumption. After that, Bacho et al. [BLT⁺24] introduced the first fully adaptive (up to $t < n$ corruptions) scheme in the pairing-free setting, called Twinkle. In contrast to the scheme by Crites et al., this scheme is even proven secure under a non-interactive assumption.

It is worth saying that these security improvements have been achieved with minimal impact on communication complexity and round efficiency, while also avoiding idealized models like the algebraic group model (AGM) [FKL18].

Concrete Security. In this work, we observe that an important dimension in terms of security guarantees has not been considered: namely, the concrete security level guaranteed by the security proof. Most prior schemes [KG20, CKM21, BTZ22, TZ23, CKM23b, Lin24, KRT24] rely on the Forking Lemma [BN06, BDL19], a technique known to yield loose security bounds. Concretely, for any adversary $\mathcal{A}$ that breaks the scheme with probability ϵ and makes at most Q random oracle and signing queries, there is a reduction that runs in approximately the same time and breaks the underlying assumption with probability ϵ' . The relation between ϵ , Q , and ϵ' is¹

$$\epsilon \leq \sqrt{\Theta(Q) \cdot \epsilon'}.$$

In other words, if we assume that the underlying assumption is κ -bit secure ($\epsilon' \leq 2^{-\kappa}$), then the scheme only guarantees a $(\kappa - \log Q)/2$ -bit security level. In practice, if the scheme is instantiated with a group providing 128-bit security, the resulting security level would be only 44 bits for $Q \approx 2^{40}$. Bacho et al. [BLT⁺24] circumvent the use of the Forking Lemma in their construction of Twinkle, allowing them to

¹In this overview, we ignore additional statistical terms that are independent of the underlying assumption when giving security bounds. We also ignore the running times of reductions and the fact that many of these schemes rely on less standard interactive assumptions [KG20, BCK⁺22, CKM23b]. Further, the recent schemes Sparkle+ [CKM23a] and KRT [KRT24] also include an additive term related loosely to the security of a standard signature scheme.

achieve full adaptive security. As a side effect, this also leads to an improvement in concrete security. Specifically, the security bound is

$$\epsilon \leq \Theta(Q) \cdot \epsilon'.$$

While this is a clear improvement, it still leads to weak concrete security guarantees. Specifically, assuming 128-bit security for DDH and $Q \approx 2^{40}$, we are left with only $128 - 40 = 88$ bits of security for Twinkle. A tightly secure scheme, i.e., a scheme with a bound $\epsilon \leq O(1) \cdot \epsilon'$ for a small constant, with the same qualitative guarantees as Twinkle remains an open problem.

1.1 Our Contribution

We solve this open problem by presenting Twinkle-T, the first tightly secure threshold signature scheme. We only rely on the random oracle model and not on any other idealizations such as generic or algebraic group models. Our scheme achieves the following:

- *Full Adaptivity.* Twinkle-T is secure in the presence of an adversary that makes up to $t < n$ adaptive corruptions.
- *Non-Interactive Assumptions.* Twinkle-T can be instantiated using the same assumptions as Twinkle, for instance, based on DDH over pairing-free prime order groups.
- *Tight Security.* In contrast to Twinkle, we can give a tight security proof for Twinkle-T.

Concretely, while Twinkle guarantees a security level of at most 88 bits over a 128-bit secure group, our scheme is 127-bit secure. At the same time, we emphasize that the efficiency of our scheme stay within a practical regime, and we achieve this without increasing the number of rounds compared to Twinkle [BLT⁺24]. That is, our signing protocol consists of only three rounds. For a comparison of threshold signature schemes in pairing-free cyclic groups, see Table 1.

We emphasize that we are the first to initiate the study of tight security for threshold signatures. In particular, while both tightness and pairing-free threshold signatures have been active areas of research, no one has looked at the combination prior to our work. Therefore, our work will open the door to further research in this area. We also remark that even in the pairing-friendly setting or the statically secure setting, we are not aware of any tightly secure construction from a standard assumption.

Scheme	Rounds	Adaptive	Assumption	Security Loss
Frost [KG20], Frost2 [BCK ⁺ 22], Frost3 [RRJ ⁺ 22]	2	✗	AOMDL	$\Theta(Q/\epsilon)$
TZ [TZ23]	2	✗	DLOG	$\Theta(Q/\epsilon)$
Lindell [Lin24]	3	✗	DLOG	$\Theta(Q/\epsilon)$
Classic-S [Mak22]	3	✗	DLOG	$\Theta(Q/\epsilon)$
Sparkle [CKM23b]	3	✓	AOMDL	$\Theta(Q/\epsilon)$
Sparkle+ [CKM23a]	3	✓	AOMDL	$\Theta(Q/\epsilon)$
Zero-S [Mak22]	3	✓	DLOG	$\Theta(Q/\epsilon)$
Twinkle _{AOMCDH} [BLT ⁺ 24]	3	✓	AOMCDH	$\Theta(Q)$
Twinkle _{DDH} [BLT ⁺ 24]	3	✓	DDH	$\Theta(Q)$
KRT [KRT24]	5	✓	DLOG	$\Theta(Q^3/\epsilon)$
Glacius [BDLR24]	5	✓	DDH	$\Theta(Q/\epsilon)$
Dazzle-T (concurrent) [Che25]	3	✓	DDH	$\Theta(1)$
Twinkle-T _{AOMCDH} (ours)	3	✓	AOMCDH	$\Theta(1)$
Twinkle-T _{DDH} (ours)	3	✓	DDH	$\Theta(1)$

Table 1: Comparison of threshold signature schemes in pairing-free cyclic groups. We assume a trusted dealer distributing key shares. We do not consider proofs in the algebraic group model (AGM), or schemes assuming a broadcast channel [CGJ⁺99, SS01, AF04, GJKR07, BHK⁺24, GS24]. We denote the total number of random oracle and signing queries by Q , and the advantage of an adversary against the scheme by ϵ . Sparkle+ and KRT also include an additive term in their security loss related loosely to the security of a standard signature scheme. For simplicity, we omit these terms, which is in favor of them. Lindell, Classic-S, and Zero-S UC-realize an ideal functionality for computing Schnorr signatures, which are known to have an inherent security loss of $\Omega(Q/\epsilon)$ [Seu12]. Zero-S assumes secure erasures and secret channels to achieve adaptive security. Sparkle and Sparkle+ have corruption threshold $t/2$, while all others have full corruption threshold $t < n$.

1.2 More on Related Work

As a starting point for further reading, we give references to relevant related work, particularly, on adaptively secure threshold signatures, other multi-signature variants, and the study of tight security.

Adaptively Secure Threshold Signatures. Initial works [CGJ⁺99, FMY99, JL00, LP01, AF04, ADN06, WQL09] on threshold cryptography use the *single inconsistent player* (SIP) technique to design adaptively secure threshold signatures. These approaches, however, require a lot of interaction rounds. Notably, none of these constructions has a tight security proof. More critically, existing impossibility results for their underlying signature schemes [Cor00, HJK12, Seu12, KK12] rule out any tight security proofs. Recent works [LJY14, BL22, DR24] establish adaptive security for BLS-type threshold signatures. Among these, only [BL22] gives a tight security proof, but relies on the AGM and the OMDL assumption. Threshold Schnorr signatures have recently gained a lot of attention and several adaptively secure schemes have been proposed [Mak22, CKM23a, CKM23b, KRT24, BLSW24]. Among these, only [CKM23b, BLSW24] give a tight security proof, again by relying on the AGM and the AOMDL assumption. Importantly, existing impossibility results for Schnorr-type signatures rule out tight security proofs in the ROM [Seu12] from standard assumptions. Adaptively secure threshold ECDSA signatures are given in [CGG⁺20], but only for the case $t = n - 1$.

Other Multi-Party Signatures. Multi-signatures are a form of threshold signatures where $t = n - 1$, meaning all n parties need to participate in the signing protocol, and each party can independently generate its keys. This allows to avoid the need of a distributed key generation (DKG) protocol, as used in threshold signatures. Since their introduction [IN83], a large amount of works have emerged [MOR01, Bol03, BN06, LOS⁺06, BCJ08, BJ10], and especially in recent years [BDN18, MPSW19, DEF⁺19, NRSW20, NRS21, AB21, BD21, BTT22, FSZ22, TZ23]. Further, some works also focus on tightness [BN06, RY07, BNN07, BJ08, QLH12, FH20, PW23, PW24, BW24, BW25]. Since there is no joint threshold public key pk in multi-signatures and adaptive security is (non-tightly) implied by static security for multi-signatures², other techniques are needed to construct tightly and adaptively secure threshold signatures. Existing approaches to obtain tightness in multi-signatures are via lossy identification [AFLT12], requiring more than one key per signer [PW23], and the Katz-Wang pseudorandom bit sampling [KW03]. Crucially, none of these techniques translates to the threshold setting with adaptive corruptions (cf. Section 2). There has recently also been an increase in works on lattice-based threshold signatures [GKS24, dPKM⁺24, EKT24, CATZ24]. Blind threshold and multi-signatures have also appeared very recently [CKM⁺23c, KRB⁺24].

Tightness in General. Tightness has also been considered for other types of signatures, e.g., structure-preserving signatures [HJ12, GHKP18, CKP⁺23, AHN⁺23], lattice- and isogeny-based signatures [EKP20, PW22], and identity-based signatures and encryption [HKS15, HJP18, KYY18, PW21, HKK⁺24].

Concurrent and Independent Work. Concurrent or subsequent to our work, Chen [Che25] proposed Dazzle-T, a tightly secure, fully adaptive 3-round threshold signature from DDH that improves upon ours in both the signature size and communication cost. Concretely, the public key of Dazzle-T consists of 2 group elements (as in Twinkle-T), the signature size is 1 group element and 3 field elements (i.e., 1 group element plus the random salt ρ shorter than in Twinkle-T), and the communication cost is 4 group elements and 2 field elements (i.e., 2 group elements plus 1 NIZK proof plus the random salt ρ shorter than in Twinkle-T). Chen achieves this result by directly improving the specific scheme Twinkle_{DDH}. His key and elegant observation is that the first hash-commitment round in Twinkle_{DDH} is unnecessary; the scheme remains secure without it (Twinkle relies on an HVZK-based security proof, while Chen uses a witness-indistinguishability-based proof). Moreover, Chen observes that the tag h in Twinkle_{DDH} can be a (2×1) -matrix rather than a (2×2) -matrix, reducing the signature size by one group element. These insights lead to a more efficient two-round scheme, Dazzle, which already achieves adaptive security.

Starting from Dazzle, Chen then constructs the fully tight, three-round Dazzle-T using a conceptually similar idea to ours: introducing high-entropy input into the random oracle that determines the tag (then placed in the g -space) for each new signing session. We achieve this via a random salt $\rho \in \{0, 1\}^\lambda$ generated in the first round, whereas Chen directly uses the list of nonces $(R_i^{(1)})_{i \in S}$ from the first round of Dazzle, whose honest contributions guarantee high entropy. This list is used to derive a random, unpredictable scalar $b \in \mathbb{Z}_p$, which rerandomizes/shifts the aggregate $\tilde{R}^{(1)} = \prod_i R_i^{(1)}$, producing an

²In the multi-signature setting, one can guess the single uncorrupted party, yielding a security loss of $1/n$ when showing adaptive security from static security. Contrary to that, the security loss would be exponential in t and n for general threshold signatures.

unpredictable aggregated nonce $R^{(1)}$ that determines the tag in the second round as $h := H_{\text{tag}}(m, R^{(1)})$. In the second round, parties compute the h -side as usual, derive the challenge c from the transcript, and output their responses σ_i in the third round.

1.3 Paper Organization

We structure this paper as follows. In Section 2, we give an informal technical overview of our results. Then, in Section 3, we introduce our notation, the definition of threshold signatures, and the definition of tagged linear function families following [BLT⁺24]. For our construction, we will use a novel efficient construction of a non-interactive argument system related to these tagged linear function families. This construction is the focus of Section 4. Then, in Section 5, we present our threshold signature construction generically from any tagged linear function family. In Section 6, we instantiate the tagged linear function families as in [BLT⁺24] and discuss the efficiency of the resulting constructions.

2 Technical Overview

In this section, we explain the challenges we face and the ideas we use to overcome them. For simplicity, we only consider an instantiation based on a one-more variant of CDH over a cyclic group $\mathbb{G}$ with generator g of prime order p . In the main body, we abstractly present the construction using tagged linear functions [BLT⁺24], which can be instantiated from DDH.

2.1 Twinkle and its Security Proof

Recall that Twinkle [BLT⁺24] is the first $(t + 1)$ -out-of- n threshold signature to achieve fully adaptive security in the random oracle model. Technically, this is accomplished by avoiding rewinding, which typically results in highly non-tight security bounds. As such, Twinkle serves as a natural starting point for our solution. While we encourage readers to refer to the technical overview in [BLT⁺24] for a deeper understanding, we will focus here on explaining the aspects of Twinkle relevant to our construction.

Twinkle Signatures. Consider a public key $\text{pk} = g^{\text{sk}}$ and public key shares $\text{pk}_i = g^{\text{sk}_i}$, where $\text{sk}_i = f(i)$ and $\text{sk} = f(0)$ for a polynomial f of degree t . A Twinkle signature for a message m has two components:

- The element $\text{pk}^{(2)} = h^{\text{sk}}$, where $h = H(\text{m})$ for random oracle H . This is computed by $t + 1$ signers revealing $\text{pk}_i^{(2)} = h^{\text{sk}_i}$. Intuitively, no t colluding signers can compute h^{sk} .
- A proof that $\text{pk}^{(2)} = h^{\text{sk}}$, which is derived from a Σ -protocol with transcript $((R^{(1)}, R^{(2)}), c, s)$, computed interactively by $t + 1$ signers in three rounds.

To securely implement this, the computation of these two parts must be interleaved as follows:

1. *Round 1.* Each signer i computes $\text{pk}_i^{(2)}$, samples a random $r_i \in \mathbb{Z}_p$, and defines nonces $R_i^{(1)} = g^{r_i}$, $R_i^{(2)} = h^{r_i}$. They send a commitment $\text{com}_i = \tilde{H}(R_i^{(1)}, R_i^{(2)}, \text{pk}_i^{(2)})$ for a random oracle $\tilde{H}$. Intuitively, this round ensures that parties choose independent nonces.
2. *Round 2.* Signers open their commitments by revealing $R_i^{(1)}, R_i^{(2)}, \text{pk}_i^{(2)}$. If any opening fails, the protocol aborts. Otherwise, $\text{pk}^{(2)}$ is computed as above, and $R^{(1)}$ (resp. $R^{(2)}$) as the product of all $R_i^{(1)}$ (resp. $R_i^{(2)}$).
3. *Round 3.* The challenge $c = \bar{H}(\text{pk}, \text{pk}^{(2)}, R^{(1)}, R^{(2)}, \text{m})$ is derived from a random oracle $\bar{H}$. Each signer computes a response s_i based on c and their (sk_i, r_i) . These responses are distributed and combined into a final response s . The signature consists of $\text{pk}^{(2)}$ and (c, s) . Intuitively (c, s) is a sound proof that $\text{pk}^{(2)} = h^{\text{sk}}$.

We refer to variables $R_i^{(1)}, R^{(1)}$ as the g -side and $\text{pk}_i^{(2)}, \text{pk}^{(2)}, R_i^{(2)}, R^{(2)}$ as the h -side of the protocol. In the following, we explain the source of the security loss of Twinkle mentioned in the introduction.

Security without Signing. We begin by assuming the adversary does not make any signing queries. The adversary is given the public key pk and all public key shares pk_i , then corrupts up to t signers,

learning their secret keys sk_i . It eventually outputs a signature $(\text{pk}^{(2)}, (c, s))$ for a message $\mathbf{m}^*$ and wins if the signature is valid. Intuitively, we want to use such an adversary to break CDH. Consider a reduction that takes a CDH challenge $(g, X, Y) \in \mathbb{G}^3$ and defines $\text{pk} := X$. The reduction sets up appropriate public key shares pk_i and runs the adversary, simulating the random oracle by defining $H(\mathbf{m}) := Y^{\delta_{\mathbf{m}}}$ for random $\delta_{\mathbf{m}}$. The adversary's forgery contains $\text{pk}^{(2)}$ and a proof that $\text{pk}^{(2)} = H(\mathbf{m}^*)^{\text{sk}} = Y^{\delta_{\mathbf{m}^*} \text{sk}}$, from which the reduction computes the CDH solution Y^{sk} using $\delta_{\mathbf{m}^*}$. However, handling adaptive corruptions is a challenge: when setting up the public key shares, the reduction can only choose up to t signers for which it knows their secret key shares. Since corruptions are adaptive, the adversary may corrupt signers the reduction has not chosen. To manage this, Twinkle employs a one-more variant³ of CDH. In this variant, the reduction receives $(g, X = \text{pk}, Y)$ and all valid public key shares pk_i ⁴. It also gets t -time oracle access to an oracle that outputs sk_i on input i . This oracle allows the reduction to handle adaptive corruptions. Importantly, the reduction must avoid rewinding to ensure it makes only t oracle queries. Otherwise, an adaptive adversary may corrupt more than t distinct parties over its two executions. For more details see [BLT⁺24].

Simulating Signing Queries – Non-Tightly. Based on what we have discussed so far, the reduction for Twinkle would be tight. The security loss arises when the reduction needs to simulate honest signers during signing interactions. This process involves two key steps:

- *Simulating the g -side.* The reduction can simulate the g -side and the responses s_i using random oracles on equivalence classes – a sophisticated technique introduced in [BLT⁺24] to avoid using a broadcast channel. We will come back to this later.
- *Simulating the h -side.* The reduction translates the g -side into the h -side, i.e., computes $(\text{pk}_i^{(2)}, R_i^{(2)})$ from $(\text{pk}_i, R_i^{(1)})$. This introduces a security loss proportional to the number of signing queries, Q .

For now, let us focus on simulating the h -side given the g -side. The reduction *partitions* the message space into two subspaces:

- *The g -space.* For messages in this space, $H(\mathbf{m}) = g^{\delta_{\mathbf{m}}}$. The reduction can simulate the h -side given the g -side and $\delta_{\mathbf{m}}$, e.g., $\text{pk}_i^{(2)} = \text{pk}_i^{\delta_{\mathbf{m}}}$.
- *The Y -space.* For these messages, $H(\mathbf{m}) = Y^{\delta_{\mathbf{m}}}$. The reduction uses a forgery on such messages to solve one-more CDH as explained above.

The reduction succeeds if all signing queries fall within the g -space, and the forgery is in the Y -space. This happens with probability $1/Q$ using well-known partitioning techniques.

2.2 Towards Tight Threshold Partitioning

Thus far, we have outlined the design and analysis of Twinkle [BLT⁺24] and discussed the reasons behind its security loss. We will now shift our focus to our solution, explaining how we can achieve tight security. For now, we will not focus on preserving the round complexity of Twinkle.

Pseudorandom Bits. In standard single-signer signatures, a prominent technique to enable tight partitioning is Katz and Wang's pseudorandom bit approach [KW03]. Here, instead of signing $\mathbf{m}$, the signer signs the pair $(\mathbf{m}, b_{\mathbf{m}})$, where $b_{\mathbf{m}} \in \{0, 1\}$ is pseudorandomly derived from the message and included in the signature. Using our terminology from above, this minimal modification allows tight partitioning: for each message $\mathbf{m}$, the reduction places $(\mathbf{m}, b_{\mathbf{m}})$ in the g -space and $(\mathbf{m}, 1 - b_{\mathbf{m}})$ in the Y -space. With this, the reduction can simulate signing queries for every message. On the other hand, as the bit $b_{\mathbf{m}^*}$ remains pseudorandom for the forgery message $\mathbf{m}^*$, the adversary forges a signature for $(\mathbf{m}^*, 1 - b_{\mathbf{m}^*})$ with probability $1/2$. In this case, the reduction can break the underlying assumption.

... and Pseudorandom Paths. Unfortunately, implementing the pseudorandom bit approach in a multi-signer setting is highly challenging, as noted in recent works [PW23, PW24] on multi-signatures. Namely, since each signer selects the pseudorandom bit $b_{\mathbf{m}}$ independently, they would generate different bits, resulting in different $h = H(\mathbf{m}, b_{\mathbf{m}})$ values, breaking the scheme's correctness. To address this, Pan

³The assumption we sketch here is slightly simplified. Also, we point out again that by phrasing the scheme abstractly and replacing $(g, x) \mapsto g^x$ with a suitable function, the interactive assumption can be avoided.

⁴To recall, valid public key shares here are exponentiated evaluations of a degree- t polynomial f with $X = g^{f(0)}$.

and Wagner [PW23, PW24] have introduced the pseudorandom paths technique. In this method, each signer holds two public keys and pseudorandomly chooses which one to use for a given message. Critically, the technique exploits that signers have independent keys, and results in exponentially many potential key combinations. Therefore, it is not at all clear how to apply this technique to threshold signatures, where a single public key represents all signers. In fact, how to use their approach to get a tightly secure multi-signature scheme with key aggregation is still an open problem.

Let’s Rewind a Bit: Ad Hoc Partitioning. It appears that the Katz-Wang technique is not well suited for our setting. Returning to the single-signer setting, an earlier approach by Bellare and Rogaway [BR96] extends the message not with a pseudorandom bit, but with a λ -bit⁵ random salt ρ . In this approach, partitioning is done in a more *ad hoc* manner: for each query $H(m, \rho)$ on a fresh pair (m, ρ) , the pair is generally placed in the Y -space. However, when signing, the reduction selects a random $\rho \in \{0, 1\}^\lambda$, assigns (m, ρ) to the g -space, and proceeds with the signature. The high entropy of ρ ensures that this pair has not been placed in the Y -space before. As the forgery message m^* has never been signed, it can only be in the Y -space and the reduction succeeds. From the single-signer perspective, it is not clear why ad hoc partitioning should be any better than the Katz-Wang technique. Clearly, it results in larger signatures, and in fact, Katz and Wang’s method was intended as an improvement. Surprisingly, however, we find that the Bellare-Rogaway technique is far more suitable for the multi-signer setting.

Ad Hoc Partitioning in a Distributed Setting. To implement ad hoc partitioning in the multi-signer setting, we must ensure that each signer contributes entropy to the selection of the message-dependent group element h . In particular, no malicious signer should be able to predict or bias the choice of h . The first prototype of our scheme proceeds as follows:

1. Each participating signer i samples a random $\rho_i \in \{0, 1\}^\lambda$ and sends ρ_i to the other signers.
2. The signers then run Twinkle, using $(m, (\rho_i)_i)$ as the message, which means that $h = H(m, (\rho_i)_i)$.

Although an adversary can choose its ρ_i based on the honest signers’ values, the tight ad hoc partitioning remains intact. Namely, when an honest signer j starts participating in the protocol for a message m , the reduction samples a random ρ_j . From that moment on, the reduction places every $(m, (\rho_i)_i)$ that is queried and contains ρ_j at the appropriate position into the g -space. Then, it sends ρ_i to the adversary, and simulates the remainder of the protocol as in Twinkle. Every other input to the random oracle is placed into the Y -space. The entropy of this single ρ_j makes the argument from above go through.

Compressing the Signature. Our prototype has a major issue: verifying the signature requires knowing *all* the ρ_i ’s, which would require including them in the signature, leading to a signature size of at least $(t + 1)\lambda$ bits. This is far from ideal, and almost as large as a trivial threshold signature⁶. To address this, we observe that the unbiasedness of the individual ρ_i ’s is only needed by the reduction during signing. In particular, the ad hoc partitioning ensures that every message that has not been signed is in the Y -space, so the individual ρ_i ’s are not necessary for tightness when it comes to the forgery. With this in mind, we can compress the ρ_i ’s into a short $\rho = \hat{H}(m, (\rho_i)_i)$, run Twinkle on message (m, ρ) , and include only ρ in the signature. The proof requires an additional step to ensure that ρ is not used before the ρ_i ’s are revealed, but the tight security still holds. As an intermediate result, we achieve a tightly secure protocol with four rounds.

2.3 Reducing the Number of Rounds

Our last goal is to reduce the number of rounds to match the round complexity of Twinkle. As it turns out, the primary technical challenge lies in simulating the g -side, rather than the h -side. To tackle this, we must modify the inner workings of Twinkle and make adjustments to Twinkle’s proof technique.

Eliminating The Commitment Round. A promising approach to reducing the number of rounds is by eliminating the first round of Twinkle, where signers send their commitments com_i . However, doing so without further modifications would allow malicious signers to bias the combined nonce $R^{(1)}$, leading to insecure schemes [DEF⁺19, NRS21]. Interestingly, recent works on multi-signatures have securely removed such an additional commitment round by utilizing homomorphic trapdoor commitments [DOTT21, PW23, PW24]. Unfortunately, multi-signatures are designed for non-adaptive security models, where there is a

⁵Throughout, λ denotes the security parameter.

⁶A trivial threshold signature would just be the concatenation of single-signer signatures from at least $t + 1$ signers.

single honest signer, and we find that this technique is incompatible with adaptive security. Specifically, in these constructions, the signing protocol for a message $\mathbf{m}$ involves a commitment key $\mathbf{ck}_m$, which is derived using a random oracle. To simulate the honest signer, the reduction must embed a trapdoor into $\mathbf{ck}_m$. Crucially, the generation of this trapdoor depends on the public key of the sole honest signer. In threshold signatures, however, there are multiple honest signers, and more critically, when generating $\mathbf{ck}_m$, there is no way to predict which signers will still be honest when $\mathbf{ck}_m$ is eventually used. While the joint public key could be used to generate the trapdoor, this would only allow to simulate final signatures, but not signature shares of honest signers.

Merging Rounds – Naively. Rather than eliminating an entire round, we could consider merging our additional ρ_i -round into the first round of Twinkle. The challenge with this approach is that the first round of Twinkle already depends on h . Specifically, signers commit to both the g -side *and the h -side* via $\text{com}_i = \tilde{H}(R_i^{(1)}, R_i^{(2)}, \text{pk}_i^{(2)})$, where $R_i^{(2)} = h^{r_i}$ and $\text{pk}_i^{(2)} = h^{\text{sk}_i}$. To compute h , however, signers first need to know all the ρ_i values. To explore a potential solution, it is helpful to understand what would go wrong if we simply omitted the h -side from the commitments, i.e., defined $\text{com}_i = \tilde{H}(R_i^{(1)})$. This would allow us to merge the two rounds, with signers computing and revealing their h -side after the merged first round. Intuitively, this naive approach introduces a security flaw: while malicious signers would be unable to bias the combined nonce $R^{(1)}$ due to the commitments, they would gain *full control* over the combined nonce $R^{(2)}$. Crucially, the adversary would know the random oracle input for the challenge $c = \tilde{H}(\text{pk}, \text{pk}^{(2)}, R^{(1)}, R^{(2)}, \mathbf{m})$ *before* the reduction. As a consequence, the reduction cannot use any random oracle programming techniques to simulate honest signers.

Twinkle’s Proof. To better understand the issues caused by naively merging the first two rounds, we must first examine how the reduction in Twinkle uses the commitments. For simplicity, consider a scenario with one honest signer i and one malicious signer j , while also ignoring $\text{pk}_i^{(2)}$ for now⁷. The reduction begins by sending a random commitment com_i . Upon receiving com_j from the malicious signer, it extracts $R_j^{(1)}$ and $R_j^{(2)}$ by searching through the random oracle queries. If no preimage is found, the malicious party will almost certainly fail to open the commitment in the next round. Once $R_j^{(1)}$ and $R_j^{(2)}$ are extracted, the reduction chooses a challenge c , samples a random response s_i , and generates $R_i^{(1)}$ and $R_i^{(2)}$ using honest-verifier zero-knowledge. It then calculates the combined nonces $R^{(1)}$ and $R^{(2)}$, and programs the random oracles so that $\text{com}_i = \tilde{H}(R_i^{(1)}, R_i^{(2)})$ and $c = \tilde{H}(\text{pk}, R^{(1)}, R^{(2)}, \mathbf{m})$. The entropy in $R^{(1)}$ ensures that the random oracles remain unprogrammed until this point. The reduction finally reveals $R_i^{(1)}$ and $R_i^{(2)}$ to the adversary and continues the protocol, relying on honest-verifier zero-knowledge to send s_i as the final response. This strategy hinges on the reduction knowing the random oracle input to $\tilde{H}$ before the adversary does. If the h -side were omitted from the commitments, the reduction could not extract $R_j^{(2)}$ and could not compute the combined nonce $R^{(2)}$. As a consequence, the reduction could not program $\tilde{H}$ accordingly, making it impossible for the reduction to generate the g -side $(R_i^{(1)}, s_i)$ without the secret key sk_i .

Our Intuition. To address this issue, we make a key observation: if $R_j^{(2)}$ is computed honestly as $R_j^{(2)} = h^{r_j}$, then it is *uniquely determined* by $R_j^{(1)} = g^{r_j}$. Even better: we know that messages we have to sign are in the g -space (cf. Section 2.1), i.e., $h = g^{\delta_m}$ for a δ_m known to the reduction. This implies that $R_j^{(2)} = h^{r_j} = (R_j^{(1)})^{\delta_m}$, allowing the reduction to compute $R_j^{(2)}$ directly from $R_j^{(1)}$. In other words, having the adversary commit to the g -side is as good as committing to both sides. Of course, this intuition relies on $R_j^{(1)}$ and $R_j^{(2)}$ sharing the same discrete logarithm with respect to g and h . To ensure the reduction only deals with this case, we need an efficient way for honest signers to verify this relationship and abort if it fails. Luckily, we can design a very efficient non-interactive zero-knowledge argument for that⁸. Concretely, signers would now only commit to the g -side, but when they open the commitments and reveal the h -side, they provide a non-interactive argument that the g -side and h -side are consistent⁹ –

⁷The complexity of Twinkle’s proof largely stems from the more intricate scenarios where multiple honest signers receive inconsistent commitments from the adversary. To handle this, the authors introduced the use of random oracles on equivalence classes, a technique we will also adopt in our proof. For the purposes of this overview, however, we can illustrate the key challenges and our solution by considering a simplified setting.

⁸Since we later aim for an instantiation from DDH, we need an efficient non-interactive argument system for general tagged linear functions (cf. Section 3.2), which we show can be designed both generically and efficiently.

⁹Note that this non-interactive argument does not have to be included in the signature.

i.e., that $R_j^{(1)}$ and $R_j^{(2)}$ share the same discrete logarithm r_j , and pk_j and $\text{pk}_j^{(2)}$ share the same discrete logarithm sk_j . In the security proof, the reduction would extract the g -side from the commitments, precompute the h -side using δ_m , and then program $\bar{H}$ as done in the Twinkle proof. Soundness guarantees that the precomputed h -side will match whatever the malicious signer sends, while zero-knowledge allows the reduction to simulate the argument without needing the discrete logarithms r_i and sk_i .

Adaptive Security Madness. We have made significant progress, and it may seem like we are finished. However, a final technical issue arises when considering adaptive corruptions. To illustrate this, consider an honest signer i . Suppose the reduction has simulated a non-interactive argument π for this signer during a signing interaction. If the adversary later corrupts signer i , the reduction must query its one-more CDH oracle to obtain sk_i and provide it to the adversary, as outlined in Section 2.1. However, the reduction also needs to provide the entire internal state of that signer. In particular, this includes the random coins that signer i would have used to generate π in the protocol. Since the proof was not computed honestly by the reduction, there is no straightforward way to retrieve these coins. This issue has already been pointed out informally in recent work [BLSW24]. Their solution involves relying on secure erasures, requiring each party to erase their random coins after computing π . As we want to avoid secure erasures, we instead rely on a non-standard notion of zero-knowledge inspired by explainable arguments [HK22]. Specifically, we require that the zero-knowledge simulator can provide well-distributed random coins for any simulated proof as soon as it learns the witness. We show that our non-interactive argument achieves this notion.

3 Preliminaries

We denote the security parameter by λ and assume that all algorithms get λ in unary as input. For a finite set S , we write $x \xleftarrow{\$} S$ to denote that x is sampled uniformly at random from S . For a probabilistic algorithm $\mathcal{A}$, we write $s := \mathcal{A}(x; \varrho)$ to denote output assignment when $\mathcal{A}$ is run on input x with random coins ϱ . If ϱ is sampled uniformly at random, we also write $s \leftarrow \mathcal{A}(x)$. Further, we write $s \in \mathcal{A}(x)$ to denote that s is a possible output of $\mathcal{A}$ on input x (i.e., there exist random coins ϱ such that $s = \mathcal{A}(x; \varrho)$). For an integer $x \in \mathbb{N}$, we define $\llbracket x \rrbracket := \{0, \dots, x\}$ and $[x] := \{1, \dots, x\}$. In all our games, if not specified otherwise, numerical variables are implicitly initialized with 0, and lists and sets are initialized with $\emptyset$. We use standard cryptographic terminology such as negligible, overwhelming, and PPT.

3.1 Threshold Signatures

We define the syntax and security of (three-round) threshold signatures with trusted key generation, following previous works [BLT⁺24].

Syntax. A (t, n) -threshold signature scheme is a tuple of PPT algorithms $\text{TS} = (\text{Setup}, \text{Gen}, \text{Sig}, \text{Ver})$ defined as follows. The setup algorithm $\text{Setup}(1^\lambda)$ outputs system parameters par , and the key generation algorithm $\text{Gen}(\text{par})$ outputs a public key pk and secret key shares $\text{sk}_1, \dots, \text{sk}_n$. Further, the signing protocol Sig is split into four algorithms $(\text{Sig}_0, \text{Sig}_1, \text{Sig}_2, \text{Combine})$. Roughly, each algorithm Sig_j specifies how a signer locally computes its protocol message pm_{j+1} for the subsequent round and updates its internal state. In more detail, $\text{Sig}_0(S, i, \text{sk}_i, m)$ takes as input the signer set S , the index of the signer $i \in [n]$, its secret key share sk_i , and the message m to be signed, and it outputs a protocol message pm_1 and a state St_1 . And Sig_j for $j \in [2]$ takes as input the signer's current state and the list $\mathcal{M}_j$ of all protocol messages from the previous round, and it outputs a protocol message pm_{j+1} and an updated state St_{j+1} . Finally, the combine algorithm $\text{Combine}(S, m, \mathcal{M}_1, \mathcal{M}_2, \mathcal{M}_3)$ converts the transcript of all protocol messages into a compact signature σ , which can then be verified using the verification algorithm $\text{Ver}(\text{pk}, m, \sigma)$. Completeness of the scheme requires for all parameters and keys, a signature generated from a signing protocol among $t + 1$ honestly behaving parties outputs a signature for which Ver outputs 1.

Definition 1 (Threshold Signature Scheme). Let $t < n$ be natural numbers. A (three-round) (t, n) -threshold signature scheme is a tuple of PPT algorithms $\text{TS} = (\text{Setup}, \text{Gen}, \text{Sig}, \text{Ver})$ with the following syntax:

- $\text{Setup}(1^\lambda) \rightarrow \text{par}$ takes as input the security parameter 1^λ and outputs public system parameters par , where par implicitly defines sets of public keys, secret keys, messages and signatures. We assume

that all other algorithms implicitly take par as input.

- $\text{Gen}(\text{par}) \rightarrow (\text{pk}, \text{sk}_1, \dots, \text{sk}_n)$ takes as input system parameters par , and outputs a public key pk and secret key shares $\text{sk}_1, \dots, \text{sk}_n$.
- $\text{Sig} = (\text{Sig}_0, \text{Sig}_1, \text{Sig}_2, \text{Combine})$ is split into four algorithms:
 - $\text{Sig}_0(S, i, \text{sk}_i, \text{m}) \rightarrow (\text{pm}_1, St_1)$ takes as input a signer set $S \subseteq [n]$, an index $i \in [n]$, a secret key share sk_i , and a message m , and outputs a protocol message pm_1 and a state St_1 .
 - $\text{Sig}_1(St_1, \mathcal{M}_1) \rightarrow (\text{pm}_2, St_2)$ takes as input a state St_1 and a tuple $\mathcal{M}_1 = (\text{pm}_{1,1}, \dots, \text{pm}_{1,l})$ of protocol messages, and outputs a protocol message pm_2 and a state St_2 .
 - $\text{Sig}_2(St_2, \mathcal{M}_2) \rightarrow \text{pm}_3$ takes as input a state St_2 and a tuple $\mathcal{M}_2 = (\text{pm}_{2,1}, \dots, \text{pm}_{2,l})$ of protocol messages, and outputs a protocol message pm_3 .
 - $\text{Combine}(S, \text{m}, \mathcal{M}_1, \mathcal{M}_2, \mathcal{M}_3) \rightarrow \sigma$ takes as input a signer set $S \subseteq [n]$, a message m , tuples $\mathcal{M}_1 = (\text{pm}_{1,1}, \dots, \text{pm}_{1,l})$, $\mathcal{M}_2 = (\text{pm}_{2,1}, \dots, \text{pm}_{2,l})$, and $\mathcal{M}_3 = (\text{pm}_{3,1}, \dots, \text{pm}_{3,l})$ of protocol messages, and outputs a signature σ .
- $\text{Ver}(\text{pk}, \text{m}, \sigma) \rightarrow b$ is deterministic, takes as input a public key pk , a message m , and a signature σ , and outputs a decision bit $b \in \{0, 1\}$.

We require that TS is complete in the following sense. For all $\text{par} \in \text{Setup}(1^\lambda)$, all $(\text{pk}, \text{sk}_1, \dots, \text{sk}_n) \in \text{Gen}(\text{par})$, all messages m , and all $S \subseteq [n]$ with $|S| = t + 1$, we have

$$\Pr [\text{Ver}(\text{pk}, \text{m}, \sigma) = 1 \mid \sigma \leftarrow \text{TS.Exec}(\text{pk}, \text{sk}_1, \dots, \text{sk}_n, S, \text{m})] = 1,$$

where algorithm TS.Exec is defined in Figure 1.

Alg $\text{TS.Exec}(\text{pk}, \text{sk}_1, \dots, \text{sk}_n, S, \text{m})$

```

01 if  $|S| \neq t + 1 \vee S \not\subseteq [n]$  : return  $\perp$ 
02 parse  $\{i_1, \dots, i_{t+1}\} := S$  s.t.  $i_1 < \dots < i_{t+1}$ 
03 for  $j \in [t + 1]$  :  $(\text{pm}_{1,i_j}, St_{1,i_j}) \leftarrow \text{Sig}_0(S, i_j, \text{sk}_{i_j}, \text{m})$ 
04  $\mathcal{M}_1 := (\text{pm}_{1,i_1}, \dots, \text{pm}_{1,i_{t+1}})$ 
05 for  $j \in [t + 1]$  :  $(\text{pm}_{2,i_j}, St_{2,i_j}) \leftarrow \text{Sig}_1(St_{1,i_j}, \mathcal{M}_1)$ 
06  $\mathcal{M}_2 := (\text{pm}_{2,i_1}, \dots, \text{pm}_{2,i_{t+1}})$ 
07 for  $j \in [t + 1]$  :  $\text{pm}_{3,i_j} \leftarrow \text{Sig}_2(St_{2,i_j}, \mathcal{M}_2)$ 
08  $\mathcal{M}_3 := (\text{pm}_{3,i_1}, \dots, \text{pm}_{3,i_{t+1}})$ 
09 return  $\sigma \leftarrow \text{Combine}(S, \text{m}, \mathcal{M}_1, \mathcal{M}_2, \mathcal{M}_3)$ 

```

Figure 1: Algorithm TS.Exec for a (three-round) (t, n) -threshold signature scheme $\text{TS} = (\text{Setup}, \text{Gen}, \text{Sig}, \text{Ver})$. The algorithm models an honest execution of the signing protocol.

Security. We define our security game following [BLT⁺24], which is an interactive version of the TS-UF-0 unforgeability notion put forth by Bellare et al. [BTZ22, BCK⁺22]. We formally define the security game in Figure 2 and give a verbal description next. The adversary gets system parameters par and an honestly generated public key pk as input. At any point in time throughout the game, the adversary can corrupt an honest party i by calling the oracle $\text{Corrupt}(i)$, for up to t parties. Upon corruption, the adversary obtains party i 's secret key sk_i and the internal state for all signing sessions party i participated in, both ongoing and completed ones. Further, the adversary can initiate a new signing session sid with some specified signer set S and message m by calling the oracle $\text{Next}(\text{sid}, S, \text{m})$. After initiating a new signing session, the adversary can interact with honest signers in these signing sessions. We model this for each signing protocol round via the signing oracles $\text{Sig}_0, \text{Sig}_1, \text{Sig}_2$. For each oracle Sig_j , the adversary can specify an honest signer i and a session identifier sid , conditioned on this signer is already in the respective round for this session sid (this is checked by an algorithm Allowed). For Sig_j with $j \in [2]$, the adversary can further specify the messages of the previous round of other signers. In particular, the adversary could send different messages to two different honest signers within the same session, and as such, we do not assume a broadcast channel. Also, the adversary could send messages to an honest signer

i on behalf of another honest signer j that deviate from what signer j actually sent, and as such, we also do not assume authenticated channels. In the end, the adversary outputs a forgery $(\mathbf{m}^*, \sigma^*)$ and wins the security game if it never started a signing session for message $\mathbf{m}^*$ and the signature σ^* is valid. We note that an interesting research direction is to achieve tight security with a stronger variant of unforgeability, in which the adversary is allowed to start signing sessions for the forgery $\mathbf{m}^*$.

On Erasures. In this security model, the private state of a signer i for a signing session sid is maintained in a map as $\text{state}[sid, i]$, which is updated after each signing round. Consequently, schemes that rely on secure erasures could satisfy this security definition. For instance, the scheme could be proven secure by requiring signers to erase part of the state from an earlier round before it gets corrupted. An example of that is the threshold signature scheme by Makriyannis [Mak22], which is proven secure using erasures. However, we emphasize that in our scheme, any state from earlier rounds can be efficiently computed from the state in the current round and the secret key share. In particular, our scheme does not rely on secure erasures.

Game $\text{TS-EUF-CMA}_{\text{TS}}^A(\lambda)$ <pre> 01 $\text{par} \leftarrow \text{Setup}(1^\lambda)$ 02 $(\text{pk}, \text{sk}_1, \dots, \text{sk}_n) \leftarrow \text{Gen}(\text{par})$ 03 $\text{Sig} := (\text{Next}, \text{Sig}_0, \text{Sig}_1, \text{Sig}_2)$ 04 $(\mathbf{m}^*, \sigma^*) \leftarrow \mathcal{A}^{\text{Sig}, \text{Corrupt}}(\text{par}, \text{pk})$ 05 if $\mathbf{m}^* \in \text{Queried}$: return 0 06 return $\text{Ver}(\text{pk}, \mathbf{m}^*, \sigma^*)$ </pre> Oracle $\text{Corrupt}(i)$ <pre> 07 if $\text{Corrupted} \geq t$: return $\perp$ 08 $\text{Corrupted} := \text{Corrupted} \cup \{i\}$ 09 return $(\text{sk}_i, \text{state}[\cdot, i])$ </pre> Oracle $\text{Next}(sid, S, m)$ <pre> 10 if $S \neq t + 1 \vee S \not\subseteq [n]$: return $\perp$ 11 if $sid \in \text{Sessions}$: return $\perp$ 12 $\text{Sessions} := \text{Sessions} \cup \{sid\}$ 13 $\text{message}[sid] := m, \text{signers}[sid] := S$ 14 $\text{Queried} := \text{Queried} \cup \{m\}$ 15 for $i \in S$: $\text{round}[sid, i] := 0$ </pre> Oracle $\text{Sig}_0(sid, i)$ <pre> 16 if $\text{Allowed}(sid, i, 0, \perp) = 0$: 17 return $\perp$ 18 $S := \text{signers}[sid], m := \text{message}[sid]$ 19 $(\text{pm}, St) \leftarrow \text{Sig}_0(S, i, \text{sk}_i, m)$ 20 $\text{pm}_1[sid, i] := \text{pm}, \text{state}[sid, i] := St$ 21 $\text{round}[sid, i] := 1$ 22 return pm </pre>	Oracle $\text{Sig}_1(sid, i, \mathcal{M}_1)$ <pre> 23 if $\text{Allowed}(sid, i, 1, \mathcal{M}_1) = 0$: 24 return $\perp$ 25 $(\text{pm}, St) \leftarrow \text{Sig}_1(\text{state}[sid, i], \mathcal{M}_1)$ 26 $\text{pm}_2[sid, i] := \text{pm}, \text{state}[sid, i] := St$ 27 $\text{round}[sid, i] := 2$ 28 return pm </pre> Oracle $\text{Sig}_2(sid, i, \mathcal{M}_2)$ <pre> 29 if $\text{Allowed}(sid, i, 2, \mathcal{M}_2) = 0$: 30 return $\perp$ 31 $\text{pm} \leftarrow \text{Sig}_2(\text{state}[sid, i], \mathcal{M}_2)$ 32 $\text{round}[sid, i] := 3$ 33 return pm </pre> Alg $\text{Allowed}(sid, i, r, \mathcal{M})$ <pre> 34 if $sid \notin \text{Sessions}$: return 0 35 $S := \text{signers}[sid], H := S \setminus \text{Corrupted}$ 36 if $i \notin H$: return 0 37 if $\text{round}[sid, i] \neq r$: return 0 38 if $r > 0$: 39 parse $(\text{pm}_i)_{i \in S} := \mathcal{M}$ 40 if $\text{pm}_i \neq \text{pm}_r[sid, i]$: return 0 41 return 1 </pre>
--	--

Figure 2: The game **TS-EUF-CMA** for a (three-round) (t, n) -threshold signature scheme $\text{TS} = (\text{Setup}, \text{Gen}, \text{Sig}, \text{Ver})$ and an adversary $\mathcal{A}$.

Definition 2 (TS-EUF-CMA Security). Let $\text{TS} = (\text{Setup}, \text{Gen}, \text{Sig}, \text{Ver})$ be a (t, n) -threshold signature scheme. Consider the game **TS-EUF-CMA** defined in Figure 2. We say that TS is TS-EUF-CMA secure, if for all PPT adversaries $\mathcal{A}$, the following advantage is negligible:

$$\text{Adv}_{\mathcal{A}, \text{TS}}^{\text{TS-EUF-CMA}}(\lambda) := \Pr \left[\text{TS-EUF-CMA}_{\text{TS}}^{\mathcal{A}}(\lambda) \Rightarrow 1 \right].$$

3.2 Tagged Linear Function Families

Following [BLT⁺24], we make use of the abstraction of tagged linear function families. Consider a field $\mathcal{S}$ (the scalars), a set $\mathcal{T}$ (the tags), and vector spaces $\mathcal{D}$ (the domain) and $\mathcal{R}$ (the range) over $\mathcal{S}$, all parameterized by some public parameters par . Then, a tagged linear function family is a tuple $\text{TLF} = (\text{Gen}, \text{T})$ of PPT algorithms, where $\text{Gen}(1^\lambda)$ generates such parameters and for given parameters, T realizes a function $\text{T}: \mathcal{T} \times \mathcal{D} \rightarrow \mathcal{R}$, such that for any fixed $g \in \mathcal{T}$, $\text{T}(g, \cdot)$ is a vector space homomorphism.

Definition 3 (Tagged Linear Function Family). A tagged linear function family (TLFF) is a tuple of PPT algorithms $\text{TLF} = (\text{Gen}, \text{T})$ with the following syntax:

- $\text{Gen}(1^\lambda) \rightarrow \text{par}$ takes as input the security parameter 1^λ and outputs parameters par . We assume that par implicitly defines the following sets: a set of scalars $\mathcal{S}_{\text{par}}$, which forms a field; a set of tags $\mathcal{T}_{\text{par}}$; a domain $\mathcal{D}_{\text{par}}$ and a range $\mathcal{R}_{\text{par}}$, where each forms a vector space over $\mathcal{S}_{\text{par}}$. If par is clear from the context, we omit the subscript par . We naturally denote the operations of these fields and vector spaces by $+$ and $\cdot$, and assume that these operations can be evaluated efficiently.
- $\text{T}(\text{par}, g, x) \rightarrow X$ is deterministic, takes as input parameters par , a tag $g \in \mathcal{T}$, and a domain element $x \in \mathcal{D}$, and outputs a range element $X \in \mathcal{R}$. For all parameters par , and for all tags $g \in \mathcal{T}$, the function $\text{T}(\text{par}, g, \cdot)$ realizes a homomorphism, i.e.,

$$\forall s \in \mathcal{S}, x, y \in \mathcal{D} : \text{T}(\text{par}, g, s \cdot x + y) = s \cdot \text{T}(\text{par}, g, x) + \text{T}(\text{par}, g, y).$$

For T , we also omit the input par if it is clear from the context.

A *regular* tagged linear function family additionally satisfies that for an overwhelming fraction of pairs (par, g) , the images of random domain elements under $\text{TLF}(g, \cdot)$ are uniform over $\mathcal{R}$. We usually denote the set of such pairs by Reg and call this the regularity set.

Definition 4 (Regular TLFF). Let $\text{TLF} = (\text{Gen}, \text{T})$ be a tagged linear function family. We say that TLF is ε_r -regular, if there is a set Reg (called the regularity set) such that the following two properties hold:

- We have

$$\Pr[(\text{par}, g) \notin \text{Reg} \mid \text{par} \leftarrow \text{Gen}(1^\lambda), g \xleftarrow{\$} \mathcal{T}] \leq \varepsilon_r.$$

- For any fixed $(\text{par}, g) \in \text{Reg}$, the following distributions are the same:

$$\{(\text{par}, g, X) \mid X \xleftarrow{\$} \mathcal{R}\} \text{ and } \{(\text{par}, g, X) \mid x \xleftarrow{\$} \mathcal{D}, X := \text{T}(\text{par}, g, x)\}.$$

A second property that we require is *translatability*. It means that there is an efficient way of setting up a tag h with a trapdoor from a given tag g , formally $(h, \text{td}) \leftarrow \text{Shift}(\text{par}, g)$, such that (1) for a random g the distribution of (g, h) is statistically close to uniform over $\mathcal{T}^2$, and (2) there are deterministic polynomial-time algorithms Translate , InvTranslate with $\text{Translate}(\text{td}, \text{T}(g, x)) = \text{T}(h, x)$ and $\text{InvTranslate}(\text{td}, \text{T}(h, x)) = \text{T}(g, x)$ for all $x \in \mathcal{D}$. That is, images under g can be translated to images under h and vice versa.

Definition 5 (Translatability). Let $\text{TLF} = (\text{Gen}, \text{T})$ be a tagged linear function family. We say that TLF is ε_t -translatable, if there is a PPT algorithm Shift and a deterministic polynomial-time algorithms Translate , InvTranslate , such that the following properties hold:

- **Well-Distributed Tags.** The statistical distance between the following distributions $\mathcal{X}_0$ and $\mathcal{X}_1$ is at most ε_t :

$$\begin{aligned} \mathcal{X}_0 &:= \{(\text{par}, g, h) \mid \text{par} \leftarrow \text{Gen}(1^\lambda), g \xleftarrow{\$} \mathcal{T}, h \xleftarrow{\$} \mathcal{T}\}, \\ \mathcal{X}_1 &:= \{(\text{par}, g, h) \mid \text{par} \leftarrow \text{Gen}(1^\lambda), g \xleftarrow{\$} \mathcal{T}, (h, \text{td}) \leftarrow \text{Shift}(\text{par}, g)\}. \end{aligned}$$

- **Translation Completeness.** For every $\text{par} \in \text{Gen}(1^\lambda)$, for any $g \in \mathcal{T}$, any $x \in \mathcal{D}$, and any $(h, \text{td}) \in \text{Shift}(\text{par}, g)$, we have

$$\text{Translate}(\text{td}, \text{T}(g, x)) = \text{T}(h, x) \text{ and } \text{InvTranslate}(\text{td}, \text{T}(h, x)) = \text{T}(g, x).$$

The central security property that tagged linear function families need to satisfy is called *t-algebraic translation resistance*. Intuitively, 0-algebraic translation resistance states that no efficient adversary that is given two uniform tags $g, h \in \mathcal{T}$ and an image $X_0 = \mathsf{T}(g, x_0)$ can compute $\mathsf{T}(h, x_0)$. Further, it asks that this even holds in an interactive one-more fashion: the adversary gets $X_i = \mathsf{T}(g, x_i)$ for $i \in [t]$ and uniform $x_i \in \mathcal{D}$, and gets t -time oracle access to an oracle $\mathsf{Inv}(\alpha_0, \dots, \alpha_t)$ that outputs the linear combination $\sum_{i=0}^t \alpha_i x_i$. The adversary wins if it outputs $t+1$ elements X'_i , $i \in [t]$, such that X'_i and X_i have the same preimage with respect to tags h and g , respectively.

Definition 6 (Algebraic Translation Resistance). Let $\mathsf{TLF} = (\mathsf{Gen}, \mathsf{T})$ be a tagged linear function family, and $t \in \mathbb{N}$ be a number. Consider the game **A-TRAN-RES** defined in Figure 3. We say that TLF is *t-algebraic translation resistant*, if for any PPT algorithm $\mathcal{A}$, the following advantage is negligible:

$$\mathsf{Adv}_{\mathcal{A}, \mathsf{TLF}}^{t\text{-A-TRAN-RES}}(\lambda) := \Pr \left[t\text{-A-TRAN-RES}_{\mathsf{TLF}}^{\mathcal{A}}(\lambda) \Rightarrow 1 \right].$$

Game $t\text{-A-TRAN-RES}_{\mathsf{TLF}}^{\mathcal{A}}(\lambda)$	Oracle $\mathsf{Inv}(\alpha_0, \dots, \alpha_t)$
01 $\mathsf{par} \leftarrow \mathsf{Gen}(1^\lambda)$	08 if $q \geq t$: return $\perp$
02 $g, h \xleftarrow{\$} \mathcal{T}, x_0, \dots, x_t \xleftarrow{\$} \mathcal{D}$	09 $q := q + 1$
03 for $i \in [t]$: $X_i := \mathsf{T}(g, x_i)$	10 $x := \sum_{i=0}^t \alpha_i x_i$
04 $(X'_i)_{i=0}^t \leftarrow \mathcal{A}^{\mathsf{Inv}}(\mathsf{par}, g, h, (X_i)_{i=0}^t)$	11 return x
05 if $\forall i \in [t] (X_i, X'_i) \in \mathsf{Im}(\mathsf{par}, g, h)$:	
06 return 1	
07 return 0	

Figure 3: Game **A-TRAN-RES** for a tagged linear function family $\mathsf{TLF} = (\mathsf{Gen}, \mathsf{T})$ and adversary $\mathcal{A}$, where $\mathsf{Im}(\mathsf{par}, g, h)$ is defined as the set of pairs $(X, X') \in \mathcal{R}^2$ such that there is a $x \in \mathcal{D}$ with $\mathsf{T}(g, x) = X$ and $\mathsf{T}(h, x) = X'$.

4 Non-Interactive Arguments for Tagged Linear Functions

In our construction, we use a non-interactive zero-knowledge argument system [BFM88]. The relations we consider are defined for any given tagged linear function family, and are a natural analogue of discrete logarithm equality. In this section, we sketch the definition of non-interactive arguments and our construction. We provide formal details in Appendix A. Notably, our construction is both generic and concretely efficient.

Explainable Non-Interactive Arguments. Consider any **NP** relation $\mathcal{R}$ of statement-witness pairs $(\mathsf{stmt}, \mathsf{witn}) \in \mathcal{R}$. Informally, a non-interactive argument system for $\mathcal{R}$, with respect to a random oracle H , is a pair $\mathsf{AS} = (\mathsf{Prove}, \mathsf{VerProof})$ of PPT algorithms. The algorithm $\mathsf{Prove}^{\mathsf{H}}$ takes a pair $(\mathsf{stmt}, \mathsf{witn}) \in \mathcal{R}$ as input and outputs a proof π . The algorithm $\mathsf{VerProof}^{\mathsf{H}}$, on input the statement stmt and the proof π , decides whether to accept or reject the proof. Completeness ensures that honestly generated proofs for $(\mathsf{stmt}, \mathsf{witn}) \in \mathcal{R}$ are accepted. We require our argument system to satisfy zero-knowledge, meaning there exists an efficient simulator that can generate simulated proofs without knowledge of the witness witn , by appropriately programming the random oracle H . Additionally, to handle adaptive corruptions while avoiding the need for secure erasures, we introduce a more nuanced extension of zero-knowledge. In this notion, the simulator, after having simulated a proof, must be able to generate valid random coins that lead to this proof after learning the witness. This property is critical for our security reduction, where the simulator may need to produce proofs on behalf of honest parties and later reconstruct their entire internal state upon corruption. Without erasures, this internal state includes the random coins used in proof generation. Following [HK22], we call this notion *explainable zero-knowledge*, see Definition 9. Lastly, we require the argument system to be weakly simulation-sound, meaning no efficient adversary can produce valid proofs for false statements, even after observing simulated proofs. In our application, we do not need to simulate proofs for invalid statements, which is why we call this notion *weak* simulation-soundness. In particular, weak simulation-soundness is implied by explainable zero-knowledge and soundness. Detailed definitions can be found in Appendix A.1.

The TLF Relation. Let $\text{TLF} = (\text{Gen}, \text{T})$ be a tagged linear function family. As in Section 3.2, we denote the scalars, domain, range, and set of tags by $\mathcal{S}, \mathcal{D}, \mathcal{R}$, and $\mathcal{T}$, respectively. For fixed parameters par and a tag $g \in \mathcal{T}$, we consider the relation $\mathcal{R}_{\text{TLF}}[\text{par}, g]$, which is defined as

$$\mathcal{R}_{\text{TLF}}[\text{par}, g] := \left\{ (\text{stmt}, \text{wtn}) \in (\mathcal{T} \times \mathcal{R}^4) \times \mathcal{D}^2 \mid \begin{array}{l} R_1 = \text{T}(g, r), R_2 = \text{T}(h, r), \\ X_1 = \text{T}(g, x), X_2 = \text{T}(h, x) \end{array} \right\}.$$

Here, $\text{stmt} = (h, R_1, R_2, X_1, X_2)$ is the statement and $\text{wtn} = (r, x)$ is the witness. Intuitively, for two tags g, h , valid statements correspond to pairs of images (R_1, R_2) and (X_1, X_2) of the same domain element x and r , respectively. For our analysis (concretely, for zero-knowledge), we will assume that TLF is ε_r -regular with regularity set Reg and that $(\text{par}, g) \in \text{Reg}$.

Construction. We construct a non-interactive argument system $\text{AS}[\text{TLF}] = (\text{Prove}, \text{VerProof})$ for relation $\mathcal{R}_{\text{TLF}}[\text{par}, g]$ with respect to a random oracle $\text{H}: \{0, 1\}^* \rightarrow \mathcal{S}$. Our starting point is the observation that the mapping $\mathcal{D} \rightarrow \mathcal{R}^2$ with $x \mapsto (X_1, X_2)$ and $r \mapsto (R_1, R_2)$ as in the relation is linear. Therefore, we can easily design a Σ -protocol via a generic template for linear functions. Via the Fiat-Shamir paradigm [FS87], we get a non-interactive argument system for the sub-statements (h, X_1, X_2) and (h, R_1, R_2) . To improve efficiency, we additionally apply a batching step to combine the two sub-statements into one, namely, into $(h, X_1 + \gamma R_1, X_2 + \gamma R_2)$ for a random $\gamma \in \mathcal{S}$. We present the construction in Figure 4. Completeness follows by inspection. The proofs of explainable zero-knowledge (Lemma 1) and soundness (Lemma 2) are postponed to Appendix A.2. Combining Lemmata 1 and 2 and Lemma 5, we also obtain weak simulation-soundness of $\text{AS}[\text{TLF}]$.

Alg Prove^H(stmt, wtn)	Alg VerProof^H(stmt, π)
01 parse $(h, R_1, R_2, X_1, X_2) := \text{stmt}$	11 parse $(h, R_1, R_2, X_1, X_2) := \text{stmt}$
02 parse $(r, x) := \text{wtn}$	12 parse $(c, z) := \pi$
03 $\gamma := \text{H}(0, \text{stmt})$	13 $\gamma := \text{H}(0, \text{stmt})$
04 $\bar{X} := (R_1 + \gamma X_1, R_2 + \gamma X_2)$	14 $\bar{X} := (R_1 + \gamma X_1, R_2 + \gamma X_2)$
05 $\bar{\text{stmt}} := (h, \bar{X})$	15 $\bar{\text{stmt}} := (h, \bar{X})$
06 $\bar{\text{wtn}} := \bar{x} := r + \gamma x \in \mathcal{D}$	16 $W := \varphi_{g,h}(z) - c\bar{X}$
07 $w \xleftarrow{\$} \mathcal{D}$, $W := \varphi_{g,h}(w) \in \mathcal{R}^2$	17 if $\text{H}(1, \bar{\text{stmt}}, W) = c$: return 1
08 $c := \text{H}(1, \bar{\text{stmt}}, W)$	18 return 0
09 $z := c\bar{x} + w \in \mathcal{D}$	
10 return $\pi := (c, z) \in \mathcal{S} \times \mathcal{D}$	

Figure 4: The non-interactive argument system $\text{AS}[\text{TLF}] = (\text{Prove}, \text{VerProof})$ for relation $\mathcal{R}_{\text{TLF}}[\text{par}, g]$ with respect to a random oracle $\text{H}: \{0, 1\}^* \rightarrow \mathcal{S}$, where $\text{TLF} = (\text{Gen}, \text{T})$ is a tagged linear function family with scalars, domain, range, and set of tags by $\mathcal{S}, \mathcal{D}, \mathcal{R}$, and $\mathcal{T}$, respectively. The function $\varphi_{g,h}: \mathcal{D} \rightarrow \mathcal{R}^2$ is defined as $w \mapsto (\text{T}(g, w), \text{T}(h, w))$.

Lemma 1. Let $\text{TLF} = (\text{Gen}, \text{T})$ be a tagged linear function family with set of scalars $\mathcal{S}$, and let $\text{H}: \{0, 1\}^* \rightarrow \mathcal{S}$ be a random oracle. Further, assume that TLF is ε_r -regular with regularity set Reg and that $(\text{par}, g) \in \text{Reg}$. Then, $\text{AS}[\text{TLF}]$ satisfies ε_{zk} -explainable zero-knowledge, with $\varepsilon_{\text{zk}} \leq QQ_{\text{H}}/|\mathcal{R}|$, where Q_{H} denotes the number of random oracle queries and Q denotes the number of queries to oracles $\text{GetProof}, \text{GetCoins}$.

Lemma 2. Let $\text{TLF} = (\text{Gen}, \text{T})$ be a tagged linear function family with set of scalars $\mathcal{S}$, and let $\text{H}: \{0, 1\}^* \rightarrow \mathcal{S}$ be a random oracle. Then, $\text{AS}[\text{TLF}]$ satisfies ε_{snd} -soundness, with $\varepsilon_{\text{snd}} \leq 2Q_{\text{H}}/|\mathcal{S}|$, where Q_{H} denotes the number of random oracle queries.

5 Our Construction

In this section, we present our construction of three-round threshold signature using the abstraction of tagged linear function families. For concrete instantiations, we refer to Section 6.

5.1 Construction

Let $\text{TLF} = (\text{Gen}, \text{T})$ be a tagged linear function family with set of scalars $\mathcal{S}$, domain $\mathcal{D}$, range $\mathcal{R}$, and set of tags $\mathcal{T}$. Let $\text{H}: \{0, 1\}^* \rightarrow \mathcal{T}$, $\tilde{\text{H}}: \{0, 1\}^* \rightarrow \mathcal{S}$, and $\tilde{\text{H}}, \hat{\text{H}}: \{0, 1\}^* \rightarrow \{0, 1\}^{2\lambda}$ be random oracles. Further, let $\text{AS}[\text{TLF}] = (\text{Prove}, \text{VerProof})$ be a non-interactive argument system for relation $\mathcal{R}_{\text{TLF}}[\text{par}, g]$ as specified in Section 4. For readability, we omit the random oracle associated to AS in our description. We construct a tightly secure three-round (t, n) -threshold signature scheme $\text{Twinkle-T}[\text{TLF}] = (\text{Setup}, \text{Gen}, \text{Sig}, \text{Ver})$. We assume that there is an implicit injection from $[n]$ into $\mathcal{S}$. Further, let $\ell_{i,S}(x) := \prod_{j \in S \setminus \{i\}} (j-x)/(j-i) \in \mathcal{S}$ denote the i -th Lagrange coefficient for all $i \in [n]$ and $S \subseteq [n]$, and let $\ell_{i,S} := \ell_{i,S}(0)$. We give a verbal description of our scheme and present it formally as pseudocode in Figure 8.

Setup and Key Generation. All parties have access to the public parameters $\text{par} \leftarrow \text{TLF.Gen}(1^\lambda)$ which define the function T , and sets $\mathcal{S}, \mathcal{T}, \mathcal{D}$, and $\mathcal{R}$, and to a random tag $g \xleftarrow{\$} \mathcal{T}$. To generate keys, elements $a_j \xleftarrow{\$} \mathcal{D}$ for $j \in [t]$ are sampled. These elements form the coefficients of a polynomial of degree t . For each $i \in [n]$, we define the key pair $(\text{pk}_i, \text{sk}_i)$ for the i -th signer as

$$\text{sk}_i := \sum_{j=0}^t a_j i^j, \quad \text{pk}_i := \text{T}(g, \text{sk}_i).$$

The joint public key is defined as $\text{pk} := \text{pk}_0 := \text{T}(g, a_0)$.

Signing Protocol. Let $S \subseteq [n]$ be a set of signers of size $t+1$. We assume that all signers are aware of the set S and a message $\text{m} \in \{0, 1\}^*$ to be signed. Then, they run the following protocol phases to compute the signature:

1. *Randomness Generation and Commitment Phase.* Each signer $i \in S$ samples $\varrho_i \xleftarrow{\$} \{0, 1\}^{2\lambda}$ and $r_i \xleftarrow{\$} \mathcal{D}$. Then, the signer computes

$$R_i^{(1)} := \text{T}(g, r_i), \quad \text{com}_i := \tilde{\text{H}}(S, i, R_i^{(1)}).$$

It sends $\text{pm}_1 := (\varrho_i, \text{com}_i)$ to the other signers.

2. *Opening and Translation Phase.* Let $\mathcal{M}_1 = (\text{pm}_{1,j})_{j \in S}$ be the list of messages output in the first round, where $\text{pm}_{1,j} = (\varrho_j, \text{com}_j)$ is sent by signer $j \in S$. First, each signer $i \in S$ computes the joint random string ϱ and derives a tag h from it. This is done by computing

$$\varrho := \hat{\text{H}}(S, \text{m}, (\varrho_j)_{j \in S}), \quad h := \text{H}(\text{m}, \varrho).$$

Then, the signer computes

$$\text{pk}_i^{(2)} := \text{T}(h, \text{sk}_i), \quad R_i^{(2)} := \text{T}(h, r_i).$$

Further, it computes a proof $\pi_i := \text{Prove}(\text{stmt}, \text{witn}_i; \rho)$ for statement $\text{stmt} := (h, R_i^{(1)}, R_i^{(2)}, \text{pk}_i, \text{pk}_i^{(2)})$ with witness $\text{witn}_i := (r_i, \text{sk}_i)$ and uniform random coins $\rho \xleftarrow{\$} \mathcal{D}$. It sends $\text{pm}_2 := (\text{pk}_i^{(2)}, R_i^{(2)}, R_i^{(1)}, \pi_i)$ to the other signers.

3. *Response Phase.* Let $\mathcal{M}_2 = (\text{pm}_{2,j})_{j \in S}$ be the list of messages output in the second round, where $\text{pm}_{2,j} = (\text{pk}_j^{(2)}, R_j^{(2)}, R_j^{(1)}, \pi_j)$ is sent by signer $j \in S$. For $j \in S$, let $\text{stmt}_j := (h, R_j^{(1)}, R_j^{(2)}, \text{pk}_j, \text{pk}_j^{(2)})$. Each signer $i \in S$ checks that $\text{com}_j = \tilde{\text{H}}(S, j, R_j^{(1)})$ and $\text{VerProof}(\text{stmt}_j, \pi_j) = 1$ holds for all $j \in S$. If one of these equations does not hold, the signer aborts. Otherwise, the signer defines

$$R^{(1)} := \sum_{j \in S} R_j^{(1)}, \quad R^{(2)} := \sum_{j \in S} R_j^{(2)}, \quad \text{pk}^{(2)} := \sum_{j \in S} \ell_{j,S} \cdot \text{pk}_j^{(2)}.$$

Then, the signer computes $c := \tilde{\text{H}}(\text{pk}, \text{pk}^{(2)}, R^{(1)}, R^{(2)}, \text{m}, \varrho)$ and

$$s_i := c \cdot \ell_{i,S} \cdot \text{sk}_i + r_i.$$

It sends $\text{pm}_3 := s_i$ to the other signers.

The signature is $\sigma := (\text{pk}^{(2)}, c, s, \varrho)$ for $s := \sum_{j \in S} s_j$ and $\varrho := \hat{H}(S, \mathbf{m}, (\varrho_j)_{j \in S})$.

Signature Verification. Let pk be a public key, let $\mathbf{m} \in \{0, 1\}^*$ be a message, and let $\sigma = (\text{pk}^{(2)}, c, s, \varrho)$ be a signature. To verify σ with respect to public key pk and message $\mathbf{m}$, one first computes $h := H(\mathbf{m}, \varrho)$, and $R^{(1)} := T(g, s) - c \cdot \text{pk}$, $R^{(2)} := T(h, s) - c \cdot \text{pk}^{(2)}$. Then, one accepts the signature (i.e., outputs 1) if and only if $c = \bar{H}(\text{pk}, \text{pk}^{(2)}, R^{(1)}, R^{(2)}, \mathbf{m}, \varrho)$.

Remark 1. If a signer misbehaves, then either the zero-knowledge proofs do not verify, or the final response s_i is not correct, both of which can be publicly and efficiently checked. The non-tight scheme **Twinkle** also has this feature. On the other hand, schemes like **KRT** [KRT24], which add random masks to achieve adaptive security, do not provide this.

5.2 Security Analysis

Completeness follows by inspection. We now turn to the security analysis.

Theorem 1. *Let $\text{TLF} = (\text{Gen}, T)$ be a tagged linear function family with set of scalars $\mathcal{S}$, range $\mathcal{R}$, and set of tags $\mathcal{T}$. Let $H: \{0, 1\}^* \rightarrow \mathcal{T}$, $\bar{H}: \{0, 1\}^* \rightarrow \mathcal{S}$, and $\hat{H}, \hat{H}: \{0, 1\}^* \rightarrow \{0, 1\}^{2\lambda}$ be random oracles. Further, let $\text{AS}[\text{TLF}] = (\text{Prove}, \text{VerProof})$ be a non-interactive argument system for $\mathcal{R}_{\text{TLF}}[\text{par}, g]$. Assume that TLF is ε_r -regular, ε_t -translatable, and t -algebraic translation resistant. Further, assume that $\text{AS}[\text{TLF}]$ satisfies ε_{zk} -explainable zero-knowledge and ε_{snd} -soundness. Then, the scheme **Twinkle-T** is **TS-EUF-CMA** secure.*

Concretely, for any PPT algorithm $\mathcal{A}$ that makes at most Q_S queries in total to oracles $\text{Sig}_0, \text{Sig}_1, \text{Sig}_2$ and at most $Q_H, Q_{\bar{H}}, Q_{\hat{H}}, Q_{\hat{H}}$ queries to oracles $H, \bar{H}, \hat{H}, \hat{H}$, respectively¹⁰, there is a PPT algorithm $\mathcal{B}$ with $\mathbf{T}(\mathcal{B}) \approx \mathbf{T}(\mathcal{A})$ and

$$\begin{aligned} \text{Adv}_{\mathcal{A}, \text{Twinkle-T}[\text{TLF}]}^{\text{TS-EUF-CMA}}(\lambda) &\leq \frac{Q_S^2 + Q_S Q_{\bar{H}}(t+1) + Q_{\hat{H}}(Q_S + Q_H) + Q_{\bar{H}}^2 + Q_{\hat{H}}^2}{2^{2\lambda}} \\ &\quad + \frac{Q_S^2(Q_S + t) + 2Q_S Q_{\bar{H}}}{|\mathcal{R}|} + \frac{Q_{\bar{H}}}{|\mathcal{S}|} + Q_H \varepsilon_t + 2\varepsilon_r \\ &\quad + 2\varepsilon_{\text{zk}} + \varepsilon_{\text{snd}} + \text{Adv}_{\mathcal{B}, \text{TLF}}^{t\text{-A-TRAN-RES}}(\lambda). \end{aligned}$$

Proof. Let $\mathcal{A}$ be an adversary against the security of $\text{TS} := \text{Twinkle-T}[\text{TLF}]$. We structure our proof as a sequence of games $\mathbf{G}_0$ to $\mathbf{G}_{11}$ and a reduction to t -algebraic translation resistance.

Game $\mathbf{G}_0$: This game is the real security game $\text{TS-EUF-CMA}_{\text{TS}}^{\mathcal{A}}$ for threshold signatures: the game samples parameters par' for TLF and a random tag $g \xleftarrow{\$} \mathcal{T}$. It also samples random coefficients $a_0, \dots, a_t \xleftarrow{\$} \mathcal{D}$ and computes the public key $\text{pk} := \text{pk}_0 := T(g, a_0)$ and secret key shares $\text{sk}_i := \sum_{j=0}^t a_j i^j$ for each $i \in [n]$. Denote the corresponding public key shares by $\text{pk}_i := T(g, \text{sk}_i)$. Then, the game runs $\mathcal{A}$ on input $\text{par} := (\text{par}', g)$ and pk with access to signing oracles, corruption oracles, and random oracles. Concretely, it gets access to random oracles $H, \bar{H}, \hat{H}$, and $\hat{H}$, which the game provides by standard lazy sampling using maps $h[\cdot], \bar{h}[\cdot], \tilde{h}[\cdot]$, and $\hat{h}[\cdot]$, respectively. The set of corrupted parties is denoted by **Corrupted**. Whenever $\mathcal{A}$ calls the signing session oracle **Next** on some valid input $(\text{sid}, S, \mathbf{m})$ (i.e., $|S| = t+1$, $S \subseteq [n]$, and $\text{sid} \notin \text{Sessions}$), the message $\mathbf{m}$ is added to the set **Queried**. Finally, the adversary outputs a forgery $(\mathbf{m}^*, \sigma^*)$ and the game outputs 1 if $\mathbf{m}^* \notin \text{Queried}$, $|\text{Corrupted}| \leq t$, and $\text{Ver}(\text{pk}, \mathbf{m}^*, \sigma^*) = 1$. By definition, we have

$$\text{Adv}_{\mathcal{A}, \text{TS}}^{\text{TS-EUF-CMA}}(\lambda) = \Pr[\mathbf{G}_0 \Rightarrow 1].$$

Before we proceed with our analysis, we assume that the adversary always makes exactly t distinct corruption queries. This is without loss of generality, since one could build a wrapper adversary that internally runs $\mathcal{A}$, but never issues a corruption query twice, and makes enough corruption queries before outputting its forgery. Clearly, the wrapper adversary has the same advantage and running time as $\mathcal{A}$.

¹⁰We adhere to the common convention (see e.g., [KMP16], Page 8) to assume that the random oracle queries made in the signing oracle are included in $Q_H, Q_{\bar{H}}, Q_{\hat{H}}, Q_{\hat{H}}$.

Game G_1 : We rule out collisions for $\tilde{H}$ (used for commitments) and $\hat{H}$ (used for joint randomness). Concretely, the game aborts if there are $x \neq x'$ such that $\tilde{h}[x] = \tilde{h}[x'] \neq \perp$ or $\hat{h}[x] = \hat{h}[x'] \neq \perp$. By the birthday bound, we have

$$|\Pr[G_0 \Rightarrow 1] - \Pr[G_1 \Rightarrow 1]| \leq \frac{Q_{\tilde{H}}^2 + Q_{\hat{H}}^2}{2^{2\lambda}}.$$

Subsequent games will make use of an algorithm $\tilde{H}^{-1}$ that on input y searches for an x such that $\tilde{h}[x] = y$. If no such x is found, the algorithm returns $\perp$. Note that the game guarantees the existence of at most one such preimage of y .

Game G_2 : In this game, we introduce two initially empty lists $\hat{\mathcal{L}}$ and $\mathcal{L}$ and change the game as follows:

- Whenever $\mathcal{A}$ calls the signing oracle Sig_0 on some valid input (sid, i) (i.e., such that $\text{Allowed}(sid, i, 0, \perp) = 1$) and the honest signer i would return its message $(\varrho_i, \text{com}_i)$, the game aborts if the random oracle $\hat{H}$ has been queried before on an input of the form $(S, \mathbf{m}, (\varrho'_j)_{j \in S})$ such that $\varrho'_i = \varrho_i$. Here, the signer set S and message $\mathbf{m}$ are defined by the session identifier sid . Otherwise, it inserts $(S, \mathbf{m}, \varrho_i, i)$ into $\hat{\mathcal{L}}$. Since ϱ_i is sampled uniformly at random from $\{0, 1\}^{2\lambda}$, the probability of abort for a fixed signing query is bounded by $Q_{\hat{H}}/2^{2\lambda}$. Thus, by a union bound over all signing queries, this abort happens with probability at most $Q_S Q_{\hat{H}}/2^{2\lambda}$.
- Whenever the random oracle $\hat{H}$ is queried on a fresh input $(S, \mathbf{m}, (\varrho_i)_{i \in S})$ such that there is a corresponding entry $(S, \mathbf{m}, \varrho_i, i) \in \hat{\mathcal{L}}$, the game samples a random $\varrho \xleftarrow{\$} \{0, 1\}^{2\lambda}$ to program $\hat{H}$ as before. Then, the game aborts if the random oracle H has been queried before with $(\mathbf{m}, \varrho)$. Otherwise, the game inserts $(\mathbf{m}, \varrho)$ into $\mathcal{L}$. Since ϱ is sampled uniformly at random from $\{0, 1\}^{2\lambda}$, the probability of this abort for any fixed query to $\hat{H}$ is at most $Q_H/2^{2\lambda}$. Thus, by a union bound over all such queries, this abort happens with probability at most $Q_{\hat{H}} Q_H/2^{2\lambda}$.

Overall, we get

$$|\Pr[G_1 \Rightarrow 1] - \Pr[G_2 \Rightarrow 1]| \leq \frac{Q_{\hat{H}}(Q_S + Q_H)}{2^{2\lambda}}.$$

Game G_3 : In this game, we change how the random oracle H is programmed. Namely, we let the game sample a random tag $h^\diamond \xleftarrow{\$} \mathcal{T}$ at the beginning, and implement H as follows:

$$H(\mathbf{m}, \varrho) := \begin{cases} \text{via Shift}(\text{par}', g), & \text{if } (\mathbf{m}, \varrho) \in \mathcal{L}, \\ \text{via Shift}(\text{par}', h^\diamond), & \text{otherwise.} \end{cases}$$

In more detail, the game does the following. For a query $H(\mathbf{m}, \varrho)$ for which the hash value $h[\mathbf{m}, \varrho]$ is not yet defined and $(\mathbf{m}, \varrho) \in \mathcal{L}$, the game samples $(h, \text{td}) \leftarrow \text{Shift}(\text{par}', g)$ and sets $h[\mathbf{m}, \varrho] := h$. Further, it stores td in a map tr as $tr[\mathbf{m}, \varrho] := \text{td}$. On the other hand, if $(\mathbf{m}, \varrho) \notin \mathcal{L}$, the game samples $(h, \text{td}) \leftarrow \text{Shift}(\text{par}', h^\diamond)$, and updates the maps $tr[\mathbf{m}, \varrho] := \text{td}$ and $h[\mathbf{m}, \varrho] := h$. Clearly, G_2 and G_3 are indistinguishable by the ε_t -translatability of TLF, applied to every random oracle query. Concretely, we have

$$|\Pr[G_2 \Rightarrow 1] - \Pr[G_3 \Rightarrow 1]| \leq Q_H \varepsilon_t.$$

Before we proceed with our sequence of games, we point out the following crucial observations: first, whenever an honest party computes the tag h during the signing protocol, we know that it has been generated as a shift of g and we know the corresponding trapdoor. As a result, we will be able to ensure that the game no longer needs secret key shares sk_i to compute secondary public key shares $\text{pk}_i^{(2)} := \text{T}(h, \text{sk}_i)$ and nonces $R_i^{(2)} := \text{T}(h, r_i)$ for signing queries. Instead, it can simulate using the g -side transcript $(\text{pk}_i, R_i^{(1)})$ and the trapdoor of the shifted tags. In the remainder of our proof, we use careful delayed random oracle programming, observability of the random oracle, and an honest-verifier zero-knowledge-style programming to simulate the remaining parts of the signing queries without sk_i . As a result, the secret key sk_i will only be needed when the adversary corrupts parties. The second observation is that for the forgery message $\mathbf{m}^*$, no honest signer has ever been queried with $\mathbf{m}^*$. Thus, $\mathbf{m}^*$ never occurs in $\mathcal{L}$. In particular, the tag in the forgery is a shift of $h^\diamond$.

Game G_4 : We let the game abort if $(\text{par}', g) \notin \text{Reg}$, where Reg is the regularity set of TLF. By ε_r -regularity of TLF, we have

$$|\Pr[G_3 \Rightarrow 1] - \Pr[G_4 \Rightarrow 1]| \leq \varepsilon_r.$$

Game $\mathbf{G}_5$: In this game, we change how the non-interactive zero-knowledge proofs π_i are computed by the signing oracle $\mathbf{Sig}_1$ for honest signers $i \in S$. Namely, we switch from honestly generated proofs π_i to simulated proofs $\tilde{\pi}_i$. Upon an adaptive corruption, we would now have to provide the random coins used for generating the proofs, as already observed in [BLSW24]. To do this, we rely on the explainable zero-knowledge notion. Concretely, whenever the game runs the zero-knowledge simulator $\mathbf{Sim}$ to simulate a proof on behalf of an honest signer i , the simulator would now also output a state St associated to this signer and the session, and the game would store this state (along with signer index i and session identifier sid). Then, if signer i gets corrupted later, the game would run $\mathbf{Sim}$ again using the state St to obtain random coins $\tilde{\rho}_i$ explaining the proof $\tilde{\pi}_i$, and it does so for every session sid for which it has simulated a proof on behalf of signer i . It then includes those random coins in the internal state that it outputs for the signer i . Clearly, $\mathbf{G}_4$ and $\mathbf{G}_5$ are indistinguishable by explainable zero-knowledge of $\mathbf{AS}$, and we get

$$|\Pr[\mathbf{G}_4 \Rightarrow 1] - \Pr[\mathbf{G}_5 \Rightarrow 1]| \leq \varepsilon_{\text{zk}}.$$

Game $\mathbf{G}_6$: In this game, we change how the secondary elements (i.e., the h -side) $\text{pk}_i^{(2)}$ and $R_i^{(2)}$ are computed by the signing oracle $\mathbf{Sig}_1$. To recall, in the opening phase of the signing protocol, the signing oracle for honest party $i \in S$ in $\mathbf{G}_5$ would compute $\text{pk}_i^{(2)} := \mathsf{T}(h, \text{sk}_i)$ and $R_i^{(2)} := \mathsf{T}(h, r_i)$, where $h := \mathsf{H}(\mathbf{m}, \varrho)$ is obtained from $\varrho := \hat{\mathsf{H}}(S, \mathbf{m}, (\varrho_j)_{j \in S})$ in the commitment phase. From this game on, $\text{pk}_i^{(2)}$ and $R_i^{(2)}$ are computed via translation from pk_i and $R_i^{(1)}$ using the trapdoor output by $\mathbf{Shift}(\text{par}, g)$. Concretely:

$$\text{pk}_i^{(2)} := \mathsf{Translate}(\text{tr}[\mathbf{m}, \varrho], \text{pk}_i), \quad R_i^{(2)} := \mathsf{Translate}(\text{tr}[\mathbf{m}, \varrho], R_i^{(1)}).$$

Note that $\mathbf{G}_3$ guarantees knowledge of the trapdoor $\text{td} = \text{tr}[\mathbf{m}, \varrho]$. In particular, this is a trapdoor for g , following the observation made after $\mathbf{G}_3$. Thus, it follows from the translation completeness of $\mathbf{TLF}$ that the view of $\mathcal{A}$ remains unchanged, and we get

$$\Pr[\mathbf{G}_5 \Rightarrow 1] = \Pr[\mathbf{G}_6 \Rightarrow 1].$$

Game $\mathbf{G}_7$: In this game, we change the signing oracle again. Concretely, we change $\mathbf{Sig}_0$ for the commitment phase and $\mathbf{Sig}_1$ for the opening phase. To recall, in the commitment phase of the signing protocol, the signing oracle for honest party $i \in S$ in $\mathbf{G}_6$ would sample a random element $r_i \xleftarrow{\$} \mathcal{D}$ and send the commitment $\text{com}_i := \hat{\mathsf{H}}(S, i, R_i^{(1)})$ for $R_i^{(1)} := \mathsf{T}(g, r_i)$. Later, in the opening phase, the oracle would compute $\text{pk}_i^{(2)}$ and $R_i^{(2)}$ via translation as explained in $\mathbf{G}_6$, and send $R_i^{(1)}, R_i^{(2)}, \text{pk}_i^{(2)}$ along with a simulated proof $\tilde{\pi}$. We change this as follows: Instead of sampling an element $r_i \xleftarrow{\$} \mathcal{D}$ and computing $R_i^{(1)}$ honestly, we let signer i send a random commitment $\text{com}_i \xleftarrow{\$} \{0, 1\}^{2\lambda}$ in the first round. Additionally, the game inserts an entry (S, i, com_i) into a list $\mathbf{SimCom}$ that keeps track of these simulated commitments. If the same commitment has been sampled by the game twice, then the game aborts: i.e., if there is already an entry $(S', i', \text{com}_i) \in \mathbf{SimCom}$ such that $(S', i') \neq (S, i)$. By the birthday bound, this event occurs only with probability $Q_S^2/2^{2\lambda}$. We identify two situations where the preimage of com_i has to be revealed: First, the game has to output $R_i^{(1)}$ in the opening phase, and second, when party i gets corrupted, the value r_i has to be given to the adversary. To make the game's behavior clear in both situations, we make a case distinction.

- Consider the opening phase or the case where party i gets corrupted before it reaches the opening phase. In that case, we let the game sample a random $r_i \xleftarrow{\$} \mathcal{D}$, compute $R_i^{(1)} := \mathsf{T}(g, r_i)$, and then check if $\hat{\mathsf{H}}(S, i, R_i^{(1)})$ is already defined. If it is already defined, then the game aborts. Otherwise, it defines $\tilde{h}[S, i, R_i^{(1)}] := \text{com}_i$ and proceeds as before: i.e., in the opening phase, it would output $R_i^{(1)}$ (along with $R_i^{(2)}, \text{pk}_i^{(2)}, \tilde{\pi}$ computed as in $\mathbf{G}_6$), and upon a corruption, it would output r_i as part of its state. We reiterate that upon a corruption, random coins for simulated proofs $\tilde{\pi}_i$ are generated by running the zero-knowledge simulator $\mathbf{Sim}$ as described in $\mathbf{G}_5$.
- Consider the case where party i gets corrupted *after* the opening phase. In that case, the game has already defined r_i (see the previous case). The game handles the corruption as before (i.e., using this particular r_i).

To bound the event of abort in the above, we use the regularity of **TLF**, which implies that $R_i^{(1)}$ is uniform over the range $\mathcal{R}$. For a fixed signing query, the probability that $(S, i, R_i^{(1)})$ matches a previous query of $\mathcal{A}$ is bounded by $Q_{\tilde{H}}/|\mathcal{R}|$. Thus, by a union bound over all signing queries, the abort happens with probability at most $Q_S Q_{\tilde{H}}/|\mathcal{R}|$. Overall, we get

$$|\Pr[\mathbf{G}_6 \Rightarrow 1] - \Pr[\mathbf{G}_7 \Rightarrow 1]| \leq \frac{Q_S Q_{\tilde{H}}}{|\mathcal{R}|} + \frac{Q_S^2}{2^{2\lambda}}.$$

In the next part of our proof, we use the technique of random oracle programming on equivalence classes from [BLT⁺24]. This technique allows to simulate signing queries later using honest-verifier zero-knowledge, even when the adversary sends inconsistent sets of commitments to different honest parties. At a high level, this technique allows to identify whether two queries $q = (sid, i, \mathcal{M}_1)$ and $q' = (sid', i', \mathcal{M}'_1)$ to **Sig₁** will result in the same combined nonce *before* all commitments in $\mathcal{M}_1$ and $\mathcal{M}'_1$ have preimages. Recall that in our case $\mathcal{M}_1 := ((\varrho_j, \text{com}_j))_{j \in S}$, which defines $\varrho = \hat{H}(S, \mathbf{m}, (\varrho_j)_{j \in S})$ and $h = H(\mathbf{m}, \varrho)$. And the combined nonce is computed as $R^{(1)} = \sum_{j \in S} R_j^{(1)}$ where $\text{com}_j = \tilde{H}(S, j, R_j^{(1)})$. To establish this, the authors define an equivalence relation $\sim$ on such queries with the following two properties. First, the equivalence relation is preserved over time. Second, as soon as all commitments in $\mathcal{M}_1$ and $\mathcal{M}'_1$ have preimages, the equivalence relation defines identical resulting combined nonces, i.e., the resulting combined nonces are the same if and only if $q \sim q'$. Assuming such an equivalence relation, the simulation is done as follows. When the reduction has to reveal the nonce $R_i^{(1)}$ of an honest party $i \in S$, it first defines $c := \text{Cl}(q)$, where $\text{Cl} := H/\sim$ is an internal random oracle modulo the relation $\sim$. Concretely, this means two equivalent triples are mapped to the same output, and this is well-defined as the relation stays consistent over time. Then, it defines $R_i^{(1)} := T(g, s_i) - c \cdot \text{pk}_i^{\ell_{i,S}}$ for a randomly sampled $s_i \xleftarrow{\$} \mathcal{D}$. On the other hand, it does not define nonces of any other honest parties at that point and thus the combined nonce may not be known yet. Instead, the random oracle programming is delayed until the combined nonce is known. We now proceed with a slightly simplified formalization of this technique and refer to the original work [BLT⁺24] for more details.

Game G₈: In this game, we introduce a list **Pending** which keeps track of honest parties i and signing sessions sid for which the game cannot yet extract preimages of all commitments sent to party i in the commitment phase. In more detail, the list contains an entry $(sid, i, \mathcal{M}_1)$ if and only if:

- (i) The signing oracle **Sig₁** has been called with valid input $(sid, i, \mathcal{M}_1)$, i.e., for this query the game did not output $\perp$ because of $\text{Allowed}(sid, i, 1, \mathcal{M}_1) = 0$. Further, at that point in time, we have: For every commitment com_j in $\mathcal{M}_1$ such that $(S, j, \text{com}_j) \notin \text{SimCom}$, the preimage $(S, j, R_j^{(1)}) := \tilde{H}^{-1}(\text{com}_j)$ is defined (and thus can be extracted), and
- (ii) there is a commitment com_j in $\mathcal{M}_1$ such that $\tilde{H}^{-1}(\text{com}_j) = \perp$.

Further, the list is dynamically updated in the following two situations: First, whenever the adversary calls the signing oracle **Sig₁** with some valid input $(sid, i, \mathcal{M}_1)$ and the first condition is satisfied. In that case, the tuple $(sid, i, \mathcal{M}_1)$ is added to the list **Pending**. Second, whenever the map $\tilde{h}[\cdot]$ is changed (i.e., during queries to $\tilde{H}$ and queries to the corruption and signing oracles). In that case, an existing entry is removed from the list **Pending** when the second condition above is not satisfied anymore. In more detail, whenever the map $\tilde{h}[\cdot]$ is changed, the following is done: Initialize an empty list **New** and run through all entries $(sid, i, \mathcal{M}_1) \in \text{Pending}$ with the following steps:

- Check if the entry still satisfies the condition (ii) above. If this is the case, keep it in **Pending**.
- Otherwise, remove it from **Pending** and proceed as follows. Let $(sid, i, \mathcal{M}_1)$ be this removed tuple. In particular, all preimages $(S, j, R_j^{(1)}) := \tilde{H}^{-1}(\text{com}_j)$ are defined and can be extracted, where S is the signer set associated with the session sid . Determine the combined nonces and secondary public key

$$R^{(1)} = \sum_{j \in S} R_j^{(1)}, \quad \bar{R}^{(2)} = \sum_{j \in S} R_j^{(2)}, \quad \bar{\text{pk}}^{(2)} = \sum_{j \in S} \ell_{j,S} \text{pk}_j^{(2)},$$

where the secondary combined nonce and public key are as

$$\bar{\text{pk}}^{(2)} := \text{Translate}(tr[\mathbf{m}, \varrho], \text{pk}), \quad \bar{R}^{(2)} := \text{Translate}(tr[\mathbf{m}, \varrho], R^{(1)}).$$

Recall that $(sid, i, \mathcal{M}_1)$ defines (m, ϱ) and that $\mathbf{G}_3$ guarantees knowledge of the trapdoor $td = tr[m, \varrho]$. And following the observation made after $\mathbf{G}_3$, this is a trapdoor for g .

- If $(S, R^{(1)}, \bar{R}^{(2)}, \bar{pk}^{(2)}, m, \varrho) \notin \text{New}$ but the value $\bar{H}(\bar{pk}, \bar{pk}^{(2)}, R^{(1)}, \bar{R}^{(2)}, m, \varrho)$ is already defined, where the message m and randomness ϱ are defined by $(sid, i, \mathcal{M}_1)$, then the game aborts.
- Otherwise, sample $\bar{h}[\bar{pk}, \bar{pk}^{(2)}, R^{(1)}, \bar{R}^{(2)}, m, \varrho] \xleftarrow{\$} \mathcal{S}$ and insert the tuple $(S, R^{(1)}, \bar{R}^{(2)}, \bar{pk}^{(2)}, m, \varrho)$ into the list **New**.

Further, we introduce an additional abort condition. Namely, the game aborts if the following event happens: Upon a random oracle query to $\tilde{H}$ for which the hash value is yet undefined and freshly sampled as $\text{com} \xleftarrow{\$} \{0, 1\}^{2\lambda}$, there is already an existing entry $(sid, i, \mathcal{M}_1) \in \text{Pending}$ such that com is in $\mathcal{M}_1$. Having said that, the game change is now fully defined. We highlight that the only difference to how [BLT⁺24] defines its game change (concretely, the way **Pending** and **New** are defined and updated) is how the secondary combined nonce $\bar{R}^{(2)}$ and public key $\bar{pk}^{(2)}$ are computed. While [BLT⁺24] obtains these values via extraction from $\tilde{H}$, we compute them via translation from $\bar{pk}$ and $R^{(1)}$ using the trapdoor $tr[m, \varrho]$ for g . Clearly, this does not affect the abort conditions defined in this game. Thus, we can directly apply the probability analysis for abort from [BLT⁺24], and get

$$|\Pr[\mathbf{G}_7 \Rightarrow 1] - \Pr[\mathbf{G}_8 \Rightarrow 1]| \leq \frac{Q_S Q_{\tilde{H}}(t+1)}{2^{2\lambda}} + \frac{Q_S Q_{\tilde{H}}}{|\mathcal{R}|}.$$

Game $\mathbf{G}_9$: In this game, we introduce two algorithms **Equiv** and **GetChal**. The first algorithm allows to group tuples $(sid, i, \mathcal{M}_1)$ that have been inserted into list **Pending** into equivalence classes, while the second algorithm behaves as a random oracle on these equivalence classes. To clarify again, the equivalence relation is defined on the set of all triples in **Pending** and all triples that already have been removed from **Pending**, but not on any other tuples. In more detail, two tuples $q = (sid, i, \mathcal{M}_1)$ and $q' = (sid', i', \mathcal{M}_1')$ are equivalent if and only if the following conditions are satisfied simultaneously:

- Let S, S' and m, m' be the signer sets and messages associated with sid and sid' , respectively. And let ϱ, ϱ' be the joint randomness obtained from $\mathcal{M}_1 := ((\varrho_j, \text{com}_j))_{j \in S}, \mathcal{M}_1' := ((\varrho'_j, \text{com}'_j))_{j \in S'}$, respectively. Then, it holds that $S = S'$, $m = m'$, and $\varrho = \varrho'$.
- Let $F \subseteq S$ be the set of indices $j \in S$ such that $\tilde{H}^{-1}(\text{com}_j) = \perp$. Analogously, let $F' \subseteq S'$ be the set of indices $j \in S'$ such that $\tilde{H}^{-1}(\text{com}'_j) = \perp$. Then, it holds that $(\text{com}_j)_{j \in F} = (\text{com}'_j)_{j \in F'}$.
- Let $G := S \setminus F$ and $G' := S' \setminus F'$, where F, F' are defined as above. We know $(S, j, R_j^{(1)}) := \tilde{H}^{-1}(\text{com}_j)$ exists for all $j \in G$. Similarly, $(S', j, R_j'^{(1)}) := \tilde{H}^{-1}(\text{com}'_j)$ exists for all $j \in G'$. We define partially combined nonces and secondary public key for $(sid, i, \mathcal{M}_1)$ as

$$\tilde{R}^{(1)} = \sum_{j \in G} R_j^{(1)}, \quad \tilde{R}^{(2)} = \sum_{j \in G} R_j^{(2)}, \quad \tilde{pk}^{(2)} = \sum_{j \in G} \ell_{j,S} \cdot pk_j^{(2)},$$

where $R_j^{(2)} = \text{Translate}(tr[m, \varrho], R_j^{(1)})$ and $pk_j^{(2)} = \text{Translate}(tr[m, \varrho], pk_j)$ for all $j \in G$ are computed via translation. Analogously, we define partially combined nonces $\tilde{R}'^{(1)}, \tilde{R}'^{(2)}$ and secondary public key $\tilde{pk}'^{(2)}$ for $(sid', i', \mathcal{M}_1')$ through G' . Then, it holds that $(\tilde{R}^{(1)}, \tilde{R}^{(2)}, \tilde{pk}^{(2)}) = (\tilde{R}'^{(1)}, \tilde{R}'^{(2)}, \tilde{pk}'^{(2)})$.

To summarize, two triples $q = (sid, i, \mathcal{M}_1)$ and $q' = (sid', i', \mathcal{M}_1')$ are equivalent if and only if their signer sets, messages, joint randomness, partially combined first nonce, partially combined translated nonce and public key, and remaining commitments match. Again, we refer to the original work [BLT⁺24] for a proof that this indeed defines an equivalence relation. With this observation, we define an algorithm **GetChal** which assigns each equivalence class a random challenge $c \xleftarrow{\$} \mathcal{S}$ in a lazy manner. In more detail, it takes as input a tuple $(sid, i, \mathcal{M}_1)$ and checks if there is a tuple $(sid', i', \mathcal{M}_1')$ in the same equivalence class (using algorithm **Equiv**) that is already assigned a challenge c . In that case, it returns this value c . Otherwise, it assigns $c \xleftarrow{\$} \mathcal{S}$ to $(sid, i, \mathcal{M}_1)$. With these two algorithms, we change the game as follows. Instead of programming the random oracle $\bar{H}$ as $\bar{h}[\bar{pk}, \bar{pk}^{(2)}, R^{(1)}, \bar{R}^{(2)}, m, \varrho] \xleftarrow{\$} \mathcal{S}$ whenever an entry $(sid, i, \mathcal{M}_1)$ is removed from the list **Pending** and no abort occurs, we define $\bar{h}[\bar{pk}, \bar{pk}^{(2)}, R^{(1)}, \bar{R}^{(2)}, m, \varrho] := \text{GetChal}(sid, i, \mathcal{M}_1)$. We

claim that this programming does not change the view of the adversary. For this, we need to show that two different inputs $x \neq x'$ to $\bar{H}$ give independently sampled outputs. Let

$$x = (\text{pk}, \bar{\text{pk}}^{(2)}, R^{(1)}, \bar{R}^{(2)}, \text{m}, \varrho), \quad x' = (\text{pk}, \bar{\text{pk}}'^{(2)}, R'^{(1)}, \bar{R}'^{(2)}, \text{m}', \varrho'),$$

and let $q := (\text{sid}, i, \mathcal{M}_1)$ and $q' := (\text{sid}', i', \mathcal{M}_1')$ be the associated entries in **Pending** that were removed. We recall that the overline symbol in $\bar{\text{pk}}^{(2)}, \bar{R}^{(2)}$ indicates that these values were obtained via translation from the g -side as introduced in **G₈**. Clearly, if q and q' were not equivalent at the time of removal of the later one, then the outputs $\bar{H}(x)$ and $\bar{H}(x')$ are independent. On the other hand, if q and q' were indeed equivalent, then we have that $\text{m} = \text{m}'$, $\varrho = \varrho'$, and $(\text{pk}^{(2)}, R^{(1)}, R^{(2)}) = (\text{pk}'^{(2)}, R'^{(1)}, R'^{(2)})$, and thus $x = x'$. As a result, we get

$$\Pr[\mathbf{G}_8 \Rightarrow 1] = \Pr[\mathbf{G}_9 \Rightarrow 1].$$

At this point, we are ready to use an honest-verifier zero-knowledge-style simulation to simulate the remaining parts of signing without secret key shares. In particular, sk_i will only be needed upon corruptions. Intuitively, we can do that because now we know the challenge (using algorithm **GetChal**) already in the opening phase before fixing honest party's nonces.

Game G₁₀: In this game, we change the signing oracle and corruption oracle. To recall, in the opening phase of the signing protocol, the signing oracle **Sig₁** for honest party $i \in S$ would sample a random element $r_i \xleftarrow{\$} \mathcal{D}$, compute $R_i^{(1)} := \text{T}(g, r_i)$, and derive $\text{pk}_i^{(2)}$ and $R_i^{(2)}$ via translation from pk_i and $R_i^{(1)}$, respectively. Later, in the response phase, the party sends $s_i := c\ell_{i,S}\text{sk}_i + r_i$ where $c := \bar{H}(\text{pk}, \text{pk}^{(2)}, R^{(1)}, R^{(2)}, \text{m}, \varrho)$ and $\text{pk}^{(2)}, R^{(1)}, R^{(2)}$ are the combined secondary public key and nonces. Further, when the party gets corrupted, it has to send r_i as part of its state. We change this as follows. Instead of computing the challenge c as above, we let signer i derive c after the commitment phase (upon receiving $(\text{sid}, i, \mathcal{M}_1)$) as follows, considering two cases. First, if $(\text{sid}, i, \mathcal{M}_1)$ has not been added to the list **Pending**, then the party sets $c := 0$. Note that in this case, there is by definition a commitment com_j in $\mathcal{M}_1$ such that $(S, j, \text{com}_j) \notin \text{SimCom}$ and the preimage $\bar{H}^{-1}(\text{com}_j)$ is not defined yet. Thus, the adversary will not be able to open the commitment com_j in the opening phase and party i will never reach the response phase for this session (see abort conditions in **G₈**). Otherwise, the party sets $\bar{c} := \text{GetChal}(\text{sid}, i, \mathcal{M}_1)$. In either case, it samples an $s_i \xleftarrow{\$} \mathcal{D}$ and computes $R_i^{(1)} := \text{T}(g, s_i) - \bar{c} \cdot \ell_{i,S} \cdot \text{pk}_i$. Then, it derives $\text{pk}_i^{(2)}$ and $R_i^{(2)}$ via translation from pk_i and $R_i^{(1)}$ as before. Later, in the response phase, it outputs s_i as its signature share. Further, when the party gets corrupted after the opening phase, it sets $r_i := s_i - \bar{c} \cdot \ell_{i,S} \cdot \text{sk}_i$. Finally, we let the game abort if $\bar{c} \neq \bar{H}(\text{pk}, \text{pk}^{(2)}, R^{(1)}, R^{(2)}, \text{m}, \varrho)$, where $\text{pk}^{(2)}, R^{(1)}, R^{(2)}$ are the combined secondary public key and combined nonces computed after the opening phase. We identify two events where the abort happens. To establish these, we assume that the signer reaches the response phase. In particular, we know that the entry $(\text{sid}, i, \mathcal{M}_1)$ has been removed from the list **Pending** at some point in time.

- (1) In the opening phase, $\mathcal{A}$ sends a message $\text{pm}_{2,j} := (\text{pk}_j^{(2)}, R_j^{(2)}, R_j^{(1)}, \pi_j)$ such that $\text{pk}_j^{(2)} \neq \bar{\text{pk}}_j^{(2)}$ or $R_j^{(2)} \neq \bar{R}_j^{(2)}$ for

$$\bar{\text{pk}}_j^{(2)} := \text{Translate}(\text{tr}[\text{m}, \varrho], \text{pk}_j), \quad \bar{R}_j^{(2)} := \text{Translate}(\text{tr}[\text{m}, \varrho], R_j^{(1)})$$

yielded, but the proof π_j still verified. Recall that $\bar{\text{pk}}_j^{(2)}$ and $\bar{R}_j^{(2)}$ were used to define the equivalence classes for **GetChal** and thus $\bar{h}[\text{pk}, \bar{\text{pk}}^{(2)}, R^{(1)}, \bar{R}^{(2)}, \text{m}, \varrho]$ (see **G₉** and **G₈**). We now argue that this event can only happen with negligible probability. For that, we first observe that in this case the statement is no longer in the language. To see this, recall our relation for the argument system in Section 4. Without loss of generality, we assume that $\text{pk}_j^{(2)} \neq \bar{\text{pk}}_j^{(2)}$. The other case (i.e., $R_j^{(2)} \neq \bar{R}_j^{(2)}$) can be handled analogously. By translation completeness of **TLF**, we know $\bar{\text{pk}}_j^{(2)} = \text{T}(h, \text{sk}_j)$ from $\text{pk}_j = \text{T}(g, \text{sk}_j)$, where h is the tag defined by $(\text{sid}, i, \mathcal{M}_1)$. At the same time, a valid proof π_j for $\mathcal{R}_{\text{TLF}}[\text{par}, g]$ tells us that there exists an $x \in \mathcal{D}$ such that $\text{pk}_j = \text{T}(g, x)$ and $\text{pk}_j^{(2)} = \text{T}(h, x)$. From this, we obtain

$$\text{T}(g, \text{sk}_j) = \text{pk}_j = \text{T}(g, x) \implies \text{T}(h, \text{sk}_j) = \text{Translate}(\text{tr}[\text{m}, \varrho], \text{pk}_j) = \text{T}(h, x).$$

But this clearly contradicts the assumption that $\text{pk}_j^{(2)} \neq \bar{\text{pk}}_j^{(2)}$, which shows that the statement is not in the language. Now, we can bound the probability of this event happening using a straightforward reduction to weak simulation-soundness of $\text{AS}[\text{TLF}]$. By Lemma 5, weak simulation-soundness is implied by soundness in conjunction with explainable zero-knowledge as $\varepsilon_{\text{wssnd}} \leq \varepsilon_{\text{xzk}} + \varepsilon_{\text{snd}}$. Thus, this event happens with probability at most $\varepsilon_{\text{xzk}} + \varepsilon_{\text{snd}}$. And if this event does not happen, then $(\bar{\text{pk}}^{(2)}, \bar{R}^{(2)}) = (\text{pk}^{(2)}, R^{(2)})$, i.e., which implies that $\bar{h}$ was programmed as $\bar{h}[\text{pk}, \text{pk}^{(2)}, R^{(1)}, R^{(2)}, \text{m}, \varrho] := \text{GetChal}(\text{sid}, i, \mathcal{M}_1)$ when $(\text{sid}, i, \mathcal{M}_1)$ was removed from **Pending** (see $\mathbf{G}_8$ and $\mathbf{G}_9$).

- (2) The value $\text{GetChal}(\text{sid}, i, \mathcal{M}_1)$ has changed over time and will not match $\bar{h}[\text{pk}, \text{pk}^{(2)}, R^{(1)}, R^{(2)}, \text{m}, \varrho]$ anymore. However, this event happens only with probability at most $Q_S^2(Q_S + t)/|\mathcal{R}|$ [BLT⁺24].

Overall, we get

$$|\Pr[\mathbf{G}_9 \Rightarrow 1] - \Pr[\mathbf{G}_{10} \Rightarrow 1]| \leq \frac{Q_S^2(Q_S + t)}{|\mathcal{R}|} + \varepsilon_{\text{xzk}} + \varepsilon_{\text{snd}}.$$

Game $\mathbf{G}_{11}$: We no longer assume that $(\text{par}', g) \in \text{Reg}$. Clearly, we have

$$|\Pr[\mathbf{G}_{10} \Rightarrow 1] - \Pr[\mathbf{G}_{11} \Rightarrow 1]| \leq \varepsilon_r.$$

It remains to bound the probability that the final game $\mathbf{G}_{11}$ outputs 1. At this stage, observe that we no longer need secret key shares sk_i to simulate signing for honest parties, and sk_i is only needed upon corruptions.

Final Claim. We now conclude the proof by claiming that

$$\Pr[\mathbf{G}_{11} \Rightarrow 1] \leq \frac{Q_{\bar{\mathbf{H}}}}{|\mathcal{S}|} + \text{Adv}_{\mathcal{B}, \text{TLF}}^{t\text{-A-TRAN-RES}}(\lambda).$$

To prove the claim, we make a case distinction regarding the final forgery $(\mathbf{m}^*, \sigma^*)$ where $\sigma^* = (\text{pk}^{*(2)}, c^*, s^*, \varrho^*)$. For this, denote $h^* := \text{H}(\mathbf{m}^*, \varrho^*)$.

- There is no $x_0 \in \mathcal{D}$ such that $\text{T}(g, x_0) = \text{pk}$ and $\text{T}(h^*, x_0) = \text{pk}^{*(2)}$. We can bound the probability of this event using Lemma 7. For this, we build a reduction $\mathcal{I}$ that runs in the game defined in Lemma 7 and succeeds (if some guess of it was correct). The reduction $\mathcal{I}$ gets as input parameters par' for TLF. Then, it samples a random $i^* \xleftarrow{\$} [Q_{\bar{\mathbf{H}}}]$ and simulates game $\mathbf{G}_{11}$ for $\mathcal{A}$ with the following change: Upon the i^* -th query $x^* := (\text{pk}, \text{pk}^{(2)}, R^{(1)}, R^{(2)}, \text{m}, \varrho)$ to random oracle $\bar{\mathbf{H}}$, $\mathcal{I}$ outputs its state along with

$$g, h := \text{H}(\text{m}, \varrho), X_1 := \text{pk}, X_2 := \text{pk}^{(2)}, R_1 := R^{(1)}, R_2 := R^{(2)}$$

to the game and obtains a challenge $c \in \mathcal{S}$. If the hash value $\bar{\mathbf{H}}(x^*)$ is already defined, the reduction aborts. Otherwise, it programs $\bar{\mathbf{H}}(x^*) := c$ and continues the simulation. At a later stage, when $\mathcal{A}$ outputs its forgery, the reduction aborts if the query defining c^* was not x^* . Otherwise, it outputs s^* to the game. Note that the random oracle is never reprogrammed at that position, as $\mathcal{A}$ is not allowed to make a signing query for $\mathbf{m}^*$. Further, it is clear that the reduction is successful against the game from Lemma 7 if the guess i^* was correct. As the view of $\mathcal{A}$ in its interaction with the reduction is independent of i^* , we can bound the probability of this case by $Q_{\bar{\mathbf{H}}}/|\mathcal{S}|$.

- There is an $x_0 \in \mathcal{D}$ such that $\text{T}(g, x_0) = \text{pk}$ and $\text{T}(h^*, x_0) = \text{pk}^{*(2)}$. We can bound the probability of this event using an efficient reduction $\mathcal{B}$ against the t -algebraic translation resistance of TLF. For this, $\mathcal{B}$ gets as input parameters par' for TLF, tags g, h , and images $X_0, \dots, X_t \in \mathcal{R}$. Then, it simulates game $\mathbf{G}_{11}$ for $\mathcal{A}$ with the following changes:
 - **Key Setup.** It sets $\text{pk}_i := X_i$ for all $i \in [t]$. In particular, the joint public key is $\text{pk} = X_0$. Further, it sets $\text{pk}_i := \sum_{j \in [t]} \ell_{j, s_0}(i) \text{pk}_j$ for all $i \in [n] \setminus [t]$. Clearly, the public keys have the same distribution as in $\mathbf{G}_{11}$.
 - **Target Tag.** It sets $h^\diamond := h$ for the randomly sampled tag introduced from game $\mathbf{G}_3$ on.

- **Corruptions.** Whenever $\mathcal{A}$ queries the corruption oracle on some honest party i , $\mathcal{B}$ queries $x_i := \text{Inv}(\ell_{0,S_0}(i), \dots, \ell_{t,S_0}(i))$ and returns $\text{sk}_i := x_i$. Clearly, the secret key sk_i is correctly distributed. Further, $\mathcal{B}$ queries the inversion oracle exactly t times, as $\mathcal{A}$ corrupts exactly t parties during the interaction. We reiterate that this allows to compute the other values in the corruption oracle as well. Concretely, the nonce r_i for a session sid can be computed as $r_i := s_i - \bar{c} \cdot \ell_{i,S} \cdot \text{sk}_i$, where s_i is the signature share and $\bar{c}$ is derived as in $\mathbf{G}_{10}$. To obtain the random coins for simulated proofs, the reduction runs the zero-knowledge simulator Sim on the witness $\text{witn}_i := (r_i, \text{sk}_i)$ and a previously by Sim output state St when it was invoked to simulate proofs (as described in $\mathbf{G}_5$).

Finally, when $\mathcal{A}$ outputs its forgery $(\mathbf{m}^*, \sigma^*)$ and game $\mathbf{G}_{11}$ outputs 1 (which $\mathcal{B}$ can check efficiently), the reduction $\mathcal{B}$ takes $\text{td}^* := \text{tr}[\mathbf{m}^*, \varrho^*]$ and computes the inverse $X'_0 := \text{InvTranslate}(\text{td}^*, \text{pk}^{*(2)})$ for the tag $h^* := \text{H}(\mathbf{m}^*, \varrho^*)$. Recall that $(h^*, \text{td}^*) \leftarrow \text{Shift}(\text{par}', h)$ because of the changes in $\mathbf{G}_3$. Then, $\mathcal{B}$ computes

$$X'_i := \text{T}(h, x_i) \quad \forall i \in \text{Corrupted}, \quad X'_i := \sum_{j \in C^*} \ell_{j,C^*}(i) \cdot X'_j \quad \forall i \in [n] \setminus \text{Corrupted},$$

where $C^* := \text{Corrupted} \cup \{0\}$ (note that this set has size $t+1$ by game $\mathbf{G}_0$), and outputs $(X'_0, \dots, X'_t)$ to the t -algebraic translation resistance game. We argue that this indeed defines a valid solution to the game: i.e., for all $i \in [t]$, there is a $z_i \in \mathcal{D}$ such that $\text{T}(g, z_i) = X_i$ and $\text{T}(h, z_i) = X'_i$. Clearly, this is true for all $i \in \text{Corrupted}$ by construction. Further, we know by assumption that there is an $x_0 \in \mathcal{D}$ such that $\text{T}(g, x_0) = \text{pk}$ and $\text{T}(h^*, x_0) = \text{pk}^{*(2)}$. Thus, by translation completeness of TLF, it follows that $X'_0 = \text{InvTranslate}(\text{td}^*, \text{pk}^{*(2)}) = \text{T}(h, x_0)$. Further, for all $i \in [t] \setminus C^*$, we have

$$X'_i = \sum_{j \in C^*} \ell_{j,C^*}(i) \cdot X'_j = \sum_{j \in C^*} \ell_{j,C^*}(i) \cdot \text{T}(h, x_j) = \text{T}\left(h, \sum_{j \in C^*} \ell_{j,C^*}(i) \cdot x_j\right).$$

On the other hand, we have

$$\text{T}\left(g, \sum_{j \in C^*} \ell_{j,C^*}(i) \cdot x_j\right) = \sum_{j \in C^*} \ell_{j,C^*}(i) \cdot \text{T}(g, x_j) = \sum_{j \in C^*} \ell_{j,C^*}(i) \cdot \text{pk}'_j = X'_i.$$

This shows our claim and we conclude the proof. $\square$

6 Instantiations and Efficiency

We have presented our scheme using the abstraction of tagged linear function families introduced in [BLT⁺24]. In particular, we can use the instantiations of tagged linear functions provided in [BLT⁺24].

Instantiation from DDH. We use the tagged linear function family $\text{TLF}_{\text{DDH}} = (\text{Gen}_{\text{DDH}}, \text{T}_{\text{DDH}})$ from [BLT⁺24] based on the DDH assumption, which is formally defined in Appendix B. We make use of the implicit notation for group elements from [EHK⁺13]. That is, we write $[\mathbf{A}] \in \mathbb{G}^{a \times b}$ for the matrix of group elements with exponents given by the matrix $\mathbf{A} \in \mathbb{Z}_p^{a \times b}$, i.e., if $\mathbf{A} = (A_{i,j})_{i \in [a], j \in [b]}$, then $[\mathbf{A}] := (g^{A_{i,j}})_{i \in [a], j \in [b]}$. The tagged linear function family $\text{TLF}_{\text{DDH}} = (\text{Gen}_{\text{DDH}}, \text{T}_{\text{DDH}})$ based on DDH is defined as follows. Let GGen be an algorithm that takes as input 1^λ and outputs the description of a group $\mathbb{G}$ of prime order p with generator $g \in \mathbb{G}$. Algorithm Gen_{DDH} runs GGen and outputs parameters $\text{par} = (\mathbb{G}, g, p)$. These parameters define the following sets of scalars, tags, and the domain and range, respectively:

$$\mathcal{S} := \mathbb{Z}_p, \quad \mathcal{T} := \mathbb{G}^{2 \times 2}, \quad \mathcal{D} := \mathbb{Z}_p^2, \quad \mathcal{R} := \mathbb{G}^2.$$

Given a tag $[\mathbf{G}] \in \mathbb{G}^{2 \times 2}$ and an input element $\mathbf{x} \in \mathbb{Z}_p^2$, the tagged linear function T_{DDH} is defined as

$$\text{T}_{\text{DDH}}([\mathbf{G}], \mathbf{x}) := [\mathbf{G}\mathbf{x}] \in \mathbb{G}^2.$$

Lemma 3 ([BLT⁺24]). *The tagged linear function family TLF_{DDH} is ε_r -regular and ε_t -translatable with $\varepsilon_r \leq (p+1)/p^2$ and $\varepsilon_t \leq (3+3p)/p^2$. Further, let $t \in \mathbb{N}$ be polynomial in λ , and assume that the DDH assumption holds relative to GGen . Then, TLF_{DDH} is t -algebraic translation resistant. Concretely, for any PPT algorithm $\mathcal{A}$, there is a PPT algorithm $\mathcal{B}$ with $\mathbf{T}(\mathcal{B}) \approx \mathbf{T}(\mathcal{A})$ and*

$$\text{Adv}_{\mathcal{A}, \text{TLF}_{\text{DDH}}}^{t\text{-A-TRAN-RES}}(\lambda) \leq \text{Adv}_{\mathcal{B}, \text{GGen}}^{\text{DDH}}(\lambda) + \frac{4}{p} + \frac{2}{p^2}.$$

Instantiation from OMCDH. We use the tagged linear function family $\text{TLF}_{\text{AOMCDH}} = (\text{Gen}_{\text{AOMCDH}}, \text{T}_{\text{AOMCDH}})$ from [BLT⁺24] based on the AOMCDH assumption. We recall the AOMCDH assumption in Appendix B. The tagged linear function family $\text{TLF}_{\text{AOMCDH}} = (\text{Gen}_{\text{AOMCDH}}, \text{T}_{\text{AOMCDH}})$ based on AOMCDH is defined as follows. Let GGen be an algorithm that takes as input 1^λ and outputs the description of a group $\mathbb{G}$ of prime order p with generator $g \in \mathbb{G}$. Algorithm $\text{Gen}_{\text{AOMCDH}}$ runs GGen and outputs parameters $\text{par} = (\mathbb{G}, g, p)$. These parameters define the following sets of scalars, tags, and the domain and range, respectively:

$$\mathcal{S} := \mathbb{Z}_p, \quad \mathcal{T} := \mathbb{G}, \quad \mathcal{D} := \mathbb{Z}_p, \quad \mathcal{R} := \mathbb{G}.$$

Given a tag $h \in \mathbb{G}$ and an input element $x \in \mathbb{Z}_p$, the tagged linear function T_{AOMCDH} is defined as

$$\text{T}_{\text{AOMCDH}}(h, x) := h^x \in \mathbb{G}.$$

Lemma 4 ([BLT⁺24]). *The tagged linear function family $\text{TLF}_{\text{AOMCDH}}$ is ε_r -regular and ε_t -translatable with $\varepsilon_r \leq 1/p$ and $\varepsilon_t \leq 2/p$. Further, let $t \in \mathbb{N}$ be polynomial in λ , and assume that the t -AOMCDH assumption holds relative to GGen . Then, $\text{TLF}_{\text{AOMCDH}}$ is t -algebraic translation resistant. Concretely, for any PPT algorithm $\mathcal{A}$, there is a PPT algorithm $\mathcal{B}$ with $\mathbf{T}(\mathcal{B}) \approx \mathbf{T}(\mathcal{A})$ and*

$$\text{Adv}_{\mathcal{A}, \text{TLF}_{\text{AOMCDH}}}^{t\text{-A-TRAN-RES}}(\lambda) \leq \text{Adv}_{\mathcal{B}, \text{GGen}}^{t\text{-AOMCDH}}(\lambda) + \frac{1}{p}.$$

Scheme	PK	SK	Sig	Comm	Sec
Frost [KG20], Frost2 [BCK ⁺ 22], Frost3 [RRJ ⁺ 22]	33	32	64	98	43
TZ [TZ23]	33	32	97	130	42
Classic-S [Mak22]	33	32	64	97	43
Sparkle [CKM23b]	33	32	64	97	43
Sparkle+ [CKM23a]	33	64	64	161	43
Zero-S [Mak22]	33	320	64	129	43
KRT [KRT24]	33	640	64	194	3
Glacius [BDLR24]	33	96	64	161	43
Twinkle _{AOMCDH} [BLT ⁺ 24]	33	32	97	163	85
Twinkle _{DDH} [BLT ⁺ 24]	66	64	162	294	85
Dazzle-T (concurrent) [Che25]	66	64	129	196	128
Twinkle-T _{AOMCDH} (ours)	33	32	129	259	127
Twinkle-T _{DDH} (ours)	66	64	194	422	127

Table 2: Concrete efficiency and security of threshold signature schemes in pairing-free cyclic groups. We compare size of public keys, size of secret keys per signer, signature size, communication cost per signer, and concrete security - all sizes are given in bytes. In KRT, each signer has $2n + 2$ secret keys where n is the total number of signers. In Zero-S, each signer has $n + 1$ secret keys, since it assumes point-to-point secret channels. We take $n = 9$ for our comparison. We omit Lindell [Lin24] from our comparison, since it uses online-extractable NIZK proofs [Fis05], which makes it significantly more expensive in both computation and communication than the other schemes. For concrete security, we take the security bounds stated in the papers, assume $Q = 2^{40}$ for the number of hash and signing queries each, and that the underlying assumption is 128-bit hard.

Efficiency. In Table 2, we compare the efficiency and concrete security level of our schemes with previous constructions. For the comparison, we assume that all schemes are instantiated with the secp256k1 curve and the SHA-256 hash function. Further, we assume that challenges c_i are sampled uniformly from

$\mathbb{Z}_p$ and have 256 bit-length, although some implementations may use challenges of 128 bit-length. For concrete security, we assume the underlying assumption is 128-bit hard and that the number of hash and signing queries each is $Q = 2^{40}$. Overall, our comparison indicates that our constructions only add a slight overhead in the communication cost and signature size compared to the state-of-the-art fully adaptive threshold signature **Twinkle**. As such, our schemes still remain highly practical and simple. In particular, given the strong security guarantees that our schemes provide, this minor efficiency overhead is acceptable.

Statistical Terms. We note that the security levels in Table 2 have been estimated without taking into account additive terms in the security bound that are purely statistical, e.g., terms of the form c/p for small constants c (such as in Lemma 3) or statistical terms involving the number of queries but not any assumption (e.g., the first term in Theorem 1). Note that for our assumption of $Q = 2^{40}$ queries, even cubic terms of the form $\Theta(Q^3)/p$ remain below 2^{-128} for $p \approx 2^{256}$. However, when increasing Q significantly above that, this becomes inappropriate, and statistical terms must be counted.

References

- [AB21] Handan Kiliç Alper and Jeffrey Burdges. Two-round trip schnorr multi-signatures via delinearized witnesses. In Tal Malkin and Chris Peikert, editors, *CRYPTO 2021, Part I*, volume 12825 of *LNCS*, pages 157–188, Virtual Event, August 2021. Springer, Cham. (Cited on Page 5.)
- [ADN06] Jesús F. Almansa, Ivan Damgård, and Jesper Buus Nielsen. Simplified threshold RSA with adaptive and proactive security. In Serge Vaudenay, editor, *EUROCRYPT 2006*, volume 4004 of *LNCS*, pages 593–611. Springer, Berlin, Heidelberg, May / June 2006. (Cited on Page 5.)
- [AF04] Masayuki Abe and Serge Fehr. Adaptively secure feldman VSS and applications to universally-composable threshold cryptography. In Matthew Franklin, editor, *CRYPTO 2004*, volume 3152 of *LNCS*, pages 317–334. Springer, Berlin, Heidelberg, August 2004. (Cited on Page 4, 5.)
- [AFLT12] Michel Abdalla, Pierre-Alain Fouque, Vadim Lyubashevsky, and Mehdi Tibouchi. Tightly-secure signatures from lossy identification schemes. In David Pointcheval and Thomas Johansson, editors, *EUROCRYPT 2012*, volume 7237 of *LNCS*, pages 572–590. Springer, Berlin, Heidelberg, April 2012. (Cited on Page 5.)
- [AHN⁺23] Masayuki Abe, Dennis Hofheinz, Ryo Nishimaki, Miyako Ohkubo, and Pan Jiaxin. Compact structure-preserving signatures with almost tight security. *Journal of Cryptology*, 36, 08 2023. (Cited on Page 5.)
- [BCJ08] Ali Bagherzandi, Jung Hee Cheon, and Stanislaw Jarecki. Multisignatures secure under the discrete logarithm assumption and a generalized forking lemma. In Peng Ning, Paul F. Syverson, and Somesh Jha, editors, *ACM CCS 2008*, pages 449–458. ACM Press, October 2008. (Cited on Page 5.)
- [BCK⁺22] Mihir Bellare, Elizabeth C. Crites, Chelsea Komlo, Mary Maller, Stefano Tessaro, and Chenzhi Zhu. Better than advertised security for non-interactive threshold signatures. In Yevgeniy Dodis and Thomas Shrimpton, editors, *CRYPTO 2022, Part IV*, volume 13510 of *LNCS*, pages 517–550. Springer, Cham, August 2022. (Cited on Page 3, 4, 11, 25.)
- [BD21] Mihir Bellare and Wei Dai. Chain reductions for multi-signatures and the HBMS scheme. In Mehdi Tibouchi and Huaxiong Wang, editors, *ASIACRYPT 2021, Part IV*, volume 13093 of *LNCS*, pages 650–678. Springer, Cham, December 2021. (Cited on Page 5.)
- [BDL19] Mihir Bellare, Wei Dai, and Lucy Li. The local forking lemma and its application to deterministic encryption. In Steven D. Galbraith and Shiho Moriai, editors, *ASIACRYPT 2019, Part III*, volume 11923 of *LNCS*, pages 607–636. Springer, Cham, December 2019. (Cited on Page 3.)
- [BDLR24] Renas Bacho, Sourav Das, Julian Loss, and Ling Ren. Glacius: Threshold schnorr signatures from DDH with full adaptive security. Cryptology ePrint Archive, Paper 2024/1628, 2024. (Cited on Page 4, 25.)
- [BDN18] Dan Boneh, Manu Drijvers, and Gregory Neven. Compact multi-signatures for smaller blockchains. In Thomas Peyrin and Steven Galbraith, editors, *ASIACRYPT 2018, Part II*, volume 11273 of *LNCS*, pages 435–464. Springer, Cham, December 2018. (Cited on Page 5.)
- [BFM88] Manuel Blum, Paul Feldman, and Silvio Micali. Non-interactive zero-knowledge and its applications (extended abstract). In *20th ACM STOC*, pages 103–112. ACM Press, May 1988. (Cited on Page 14.)
- [BFP21] Balthazar Bauer, Georg Fuchsbauer, and Antoine Plouviez. The one-more discrete logarithm assumption in the generic group model. In Mehdi Tibouchi and Huaxiong Wang, editors, *ASIACRYPT 2021, Part IV*, volume 13093 of *LNCS*, pages 587–617. Springer, Cham, December 2021. (Cited on Page 40.)

- [BHK⁺24] Fabrice Benhamouda, Shai Halevi, Hugo Krawczyk, Yiping Ma, and Tal Rabin. SPRINT: High-throughput robust distributed Schnorr signatures. In Marc Joye and Gregor Leander, editors, *EUROCRYPT 2024, Part V*, volume 14655 of *LNCS*, pages 62–91. Springer, Cham, May 2024. (Cited on Page 4.)
- [BJ08] Ali Bagherzandi and Stanislaw Jarecki. Multisignatures using proofs of secret key possession, as secure as the Diffie-Hellman problem. In Rafail Ostrovsky, Roberto De Prisco, and Ivan Visconti, editors, *SCN 08*, volume 5229 of *LNCS*, pages 218–235. Springer, Berlin, Heidelberg, September 2008. (Cited on Page 5.)
- [BJ10] Ali Bagherzandi and Stanislaw Jarecki. Identity-based aggregate and multi-signature schemes based on RSA. In Phong Q. Nguyen and David Pointcheval, editors, *PKC 2010*, volume 6056 of *LNCS*, pages 480–498. Springer, Berlin, Heidelberg, May 2010. (Cited on Page 5.)
- [BL22] Renas Bacho and Julian Loss. On the adaptive security of the threshold BLS signature scheme. In Heng Yin, Angelos Stavrou, Cas Cremers, and Elaine Shi, editors, *ACM CCS 2022*, pages 193–207. ACM Press, November 2022. (Cited on Page 5.)
- [BLSW24] Renas Bacho, Julian Loss, Gilad Stern, and Benedikt Wagner. HARTS: High-threshold, adaptively secure, and robust threshold schnorr signatures. Cryptology ePrint Archive, Paper 2024/280, 2024. (Cited on Page 5, 10, 19.)
- [BLT⁺24] Renas Bacho, Julian Loss, Stefano Tessaro, Benedikt Wagner, and Chenzhi Zhu. Twinkle: Threshold signatures from DDH with full adaptive security. In Marc Joye and Gregor Leander, editors, *EUROCRYPT 2024, Part I*, volume 14651 of *LNCS*, pages 429–459. Springer, Cham, May 2024. (Cited on Page 3, 4, 6, 7, 10, 11, 13, 20, 21, 23, 24, 25, 37, 40.)
- [BN06] Mihir Bellare and Gregory Neven. Multi-signatures in the plain public-key model and a general forking lemma. In Ari Juels, Rebecca N. Wright, and Sabrina De Capitani di Vimercati, editors, *ACM CCS 2006*, pages 390–399. ACM Press, October / November 2006. (Cited on Page 3, 5.)
- [BNN07] Mihir Bellare, Chanathip Namprempre, and Gregory Neven. Unrestricted aggregate signatures. In Lars Arge, Christian Cachin, Tomasz Jurdzinski, and Andrzej Tarlecki, editors, *ICALP 2007*, volume 4596 of *LNCS*, pages 411–422. Springer, Berlin, Heidelberg, July 2007. (Cited on Page 5.)
- [Bol03] Alexandra Boldyreva. Threshold signatures, multisignatures and blind signatures based on the gap-Diffie-Hellman-group signature scheme. In Yvo Desmedt, editor, *PKC 2003*, volume 2567 of *LNCS*, pages 31–46. Springer, Berlin, Heidelberg, January 2003. (Cited on Page 5.)
- [BP22] Luís T. A. N. Brandão and Rene Peralta. NIST IR 8214C: First call for multi-party threshold schemes. <https://csrc.nist.gov/pubs/ir/8214/c/ipd>, 2022. Accessed: 2024-09-17. (Cited on Page 3.)
- [BR96] Mihir Bellare and Phillip Rogaway. The exact security of digital signatures: How to sign with RSA and Rabin. In Ueli M. Maurer, editor, *EUROCRYPT’96*, volume 1070 of *LNCS*, pages 399–416. Springer, Berlin, Heidelberg, May 1996. (Cited on Page 8.)
- [BTT22] Cecilia Boschini, Akira Takahashi, and Mehdi Tibouchi. MuSig-L: Lattice-based multi-signature with single-round online phase. In Yevgeniy Dodis and Thomas Shrimpton, editors, *CRYPTO 2022, Part II*, volume 13508 of *LNCS*, pages 276–305. Springer, Cham, August 2022. (Cited on Page 5.)
- [BTZ22] Mihir Bellare, Stefano Tessaro, and Chenzhi Zhu. Stronger security for non-interactive threshold signatures: BLS and FROST. Cryptology ePrint Archive, Report 2022/833, 2022. (Cited on Page 3, 11.)
- [BW24] Renas Bacho and Benedikt Wagner. Tightly secure non-interactive BLS multi-signatures. Cryptology ePrint Archive, Paper 2024/1368, 2024. (Cited on Page 5.)

- [BW25] Renas Bacho and Benedikt Wagner. T-spoon: Tightly secure two-round multi-signatures with key aggregation. Cryptology ePrint Archive, Paper 2025/840, 2025. (Cited on Page 5.)
- [CATZ24] Rutchathon Chairattana-Apirom, Stefano Tessaro, and Chenzhi Zhu. Partially non-interactive two-round lattice-based threshold signatures. Cryptology ePrint Archive, Paper 2024/467, 2024. (Cited on Page 5.)
- [CGG⁺20] Ran Canetti, Rosario Gennaro, Steven Goldfeder, Nikolaos Makriyannis, and Udi Peled. UC non-interactive, proactive, threshold ECDSA with identifiable aborts. In Jay Ligatti, Xinming Ou, Jonathan Katz, and Giovanni Vigna, editors, *ACM CCS 2020*, pages 1769–1787. ACM Press, November 2020. (Cited on Page 5.)
- [CGJ⁺99] Ran Canetti, Rosario Gennaro, Stanislaw Jarecki, Hugo Krawczyk, and Tal Rabin. Adaptive security for threshold cryptosystems. In Michael J. Wiener, editor, *CRYPTO’99*, volume 1666 of *LNCS*, pages 98–115. Springer, Berlin, Heidelberg, August 1999. (Cited on Page 4, 5.)
- [Che25] Yanbo Chen. Dazzle: Improved adaptive threshold signatures from DDH. Cryptology ePrint Archive, Paper 2025/264, 2025. (Cited on Page 4, 5, 25.)
- [CKM21] Elizabeth Crites, Chelsea Komlo, and Mary Maller. How to prove schnorr assuming schnorr: Security of multi- and threshold signatures. Cryptology ePrint Archive, Report 2021/1375, 2021. (Cited on Page 3.)
- [CKM23a] Elizabeth Crites, Chelsea Komlo, and Mary Maller. Fully adaptive schnorr threshold signatures. Cryptology ePrint Archive, Paper 2023/445, 2023. (Cited on Page 3, 4, 5, 25.)
- [CKM23b] Elizabeth C. Crites, Chelsea Komlo, and Mary Maller. Fully adaptive Schnorr threshold signatures. In Helena Handschuh and Anna Lysyanskaya, editors, *CRYPTO 2023, Part I*, volume 14081 of *LNCS*, pages 678–709. Springer, Cham, August 2023. (Cited on Page 3, 4, 5, 25.)
- [CKM⁺23c] Elizabeth C. Crites, Chelsea Komlo, Mary Maller, Stefano Tessaro, and Chenzhi Zhu. Snowblind: A threshold blind signature in pairing-free groups. In Helena Handschuh and Anna Lysyanskaya, editors, *CRYPTO 2023, Part I*, volume 14081 of *LNCS*, pages 710–742. Springer, Cham, August 2023. (Cited on Page 5.)
- [CKP⁺23] Elizabeth C. Crites, Markulf Kohlweiss, Bart Preneel, Mahdi Sedaghat, and Daniel Slamanig. Threshold structure-preserving signatures. In Jian Guo and Ron Steinfeld, editors, *ASIACRYPT 2023, Part II*, volume 14439 of *LNCS*, pages 348–382. Springer, Singapore, December 2023. (Cited on Page 5.)
- [Cor00] Jean-Sébastien Coron. On the exact security of full domain hash. In Mihir Bellare, editor, *CRYPTO 2000*, volume 1880 of *LNCS*, pages 229–235. Springer, Berlin, Heidelberg, August 2000. (Cited on Page 5.)
- [DEF⁺19] Manu Drijvers, Kasra Edalatnejad, Bryan Ford, Eike Kiltz, Julian Loss, Gregory Neven, and Igors Stepanovs. On the security of two-round multi-signatures. In *2019 IEEE Symposium on Security and Privacy*, pages 1084–1101. IEEE Computer Society Press, May 2019. (Cited on Page 5, 8.)
- [Des88] Yvo Desmedt. Society and group oriented cryptography: A new concept. In Carl Pomerance, editor, *CRYPTO’87*, volume 293 of *LNCS*, pages 120–127. Springer, Berlin, Heidelberg, August 1988. (Cited on Page 3.)
- [DF90] Yvo Desmedt and Yair Frankel. Threshold cryptosystems. In Gilles Brassard, editor, *CRYPTO’89*, volume 435 of *LNCS*, pages 307–315. Springer, New York, August 1990. (Cited on Page 3.)

- [DOTT21] Ivan Damgård, Claudio Orlandi, Akira Takahashi, and Mehdi Tibouchi. Two-round n-out-of-n and multi-signatures and trapdoor commitment from lattices. In Juan Garay, editor, *PKC 2021, Part I*, volume 12710 of *LNCS*, pages 99–130. Springer, Cham, May 2021. (Cited on Page 8.)
- [dPKM⁺24] Rafael del Pino, Shuichi Katsumata, Mary Maller, Fabrice Mouhartem, Thomas Prest, and Markku-Juhani Saarinen. Threshold raccoon: Practical threshold signatures from standard lattice assumptions. In Marc Joye and Gregor Leander, editors, *Advances in Cryptology – EUROCRYPT 2024*, pages 219–248, Cham, 2024. Springer Nature Switzerland. (Cited on Page 5.)
- [DR24] Sourav Das and Ling Ren. Adaptively secure BLS threshold signatures from DDH and co-CDH. In Leonid Reyzin and Douglas Stebila, editors, *CRYPTO 2024, Part VII*, volume 14926 of *LNCS*, pages 251–284. Springer, Cham, August 2024. (Cited on Page 5.)
- [EHK⁺13] Alex Escala, Gottfried Herold, Eike Kiltz, Carla Ràfols, and Jorge Villar. An algebraic framework for Diffie-Hellman assumptions. In Ran Canetti and Juan A. Garay, editors, *CRYPTO 2013, Part II*, volume 8043 of *LNCS*, pages 129–147. Springer, Berlin, Heidelberg, August 2013. (Cited on Page 24.)
- [EKP20] Ali El Kaafarani, Shuichi Katsumata, and Federico Pintore. Lossy CSI-FiSh: Efficient signature scheme with tight reduction to decisional CSIDH-512. In Aggelos Kiayias, Markulf Kohlweiss, Petros Wallden, and Vassilis Zikas, editors, *PKC 2020, Part II*, volume 12111 of *LNCS*, pages 157–186. Springer, Cham, May 2020. (Cited on Page 5.)
- [EKT24] Thomas Espitau, Shuichi Katsumata, and Kaoru Takemure. Two-round threshold signature from algebraic one-more learning with errors. In Leonid Reyzin and Douglas Stebila, editors, *CRYPTO 2024, Part VII*, volume 14926 of *LNCS*, pages 387–424. Springer, Cham, August 2024. (Cited on Page 5.)
- [FH20] Masayuki Fukumitsu and Shingo Hasegawa. A tightly secure ddh-based multisignature with public-key aggregation. In *2020 Eighth International Symposium on Computing and Networking Workshops (CANDARW)*, pages 321–327, 2020. (Cited on Page 5.)
- [Fis05] Marc Fischlin. Communication-efficient non-interactive proofs of knowledge with online extractors. In Victor Shoup, editor, *CRYPTO 2005*, volume 3621 of *LNCS*, pages 152–168. Springer, Berlin, Heidelberg, August 2005. (Cited on Page 25.)
- [FKL18] Georg Fuchsbauer, Eike Kiltz, and Julian Loss. The algebraic group model and its applications. In Hovav Shacham and Alexandra Boldyreva, editors, *CRYPTO 2018, Part II*, volume 10992 of *LNCS*, pages 33–62. Springer, Cham, August 2018. (Cited on Page 3.)
- [FMY99] Yair Frankel, Philip MacKenzie, and Moti Yung. Adaptively-secure distributed public-key systems. In Jaroslav Nešetřil, editor, *Algorithms - ESA ’99*, pages 4–27, Berlin, Heidelberg, 1999. Springer Berlin Heidelberg. (Cited on Page 5.)
- [FS87] Amos Fiat and Adi Shamir. How to prove yourself: Practical solutions to identification and signature problems. In Andrew M. Odlyzko, editor, *CRYPTO’86*, volume 263 of *LNCS*, pages 186–194. Springer, Berlin, Heidelberg, August 1987. (Cited on Page 15.)
- [FSZ22] Nils Fleischhacker, Mark Simkin, and Zhenfei Zhang. Squirrel: Efficient synchronized multi-signatures from lattices. In Heng Yin, Angelos Stavrou, Cas Cremers, and Elaine Shi, editors, *ACM CCS 2022*, pages 1109–1123. ACM Press, November 2022. (Cited on Page 5.)
- [GHKP18] Romain Gay, Dennis Hofheinz, Lisa Kohl, and Jiaxin Pan. More efficient (almost) tightly secure structure-preserving signatures. In Jesper Buus Nielsen and Vincent Rijmen, editors, *EUROCRYPT 2018, Part II*, volume 10821 of *LNCS*, pages 230–258. Springer, Cham, April / May 2018. (Cited on Page 5.)

- [GJKR07] Rosario Gennaro, Stanislaw Jarecki, Hugo Krawczyk, and Tal Rabin. Secure distributed key generation for discrete-log based cryptosystems. *Journal of Cryptology*, 20(1):51–83, January 2007. (Cited on Page 4.)
- [GKS24] Kamil Doruk Gur, Jonathan Katz, and Tjerand Silde. Two-round threshold lattice-based signatures from threshold homomorphic encryption. In *Post-Quantum Cryptography: 15th International Workshop, PQCrypto 2024, Oxford, UK, June 12–14, 2024, Proceedings, Part II*, page 266–300, Berlin, Heidelberg, 2024. Springer-Verlag. (Cited on Page 5.)
- [GS24] Jens Groth and Victor Shoup. Fast batched asynchronous distributed key generation. In Marc Joye and Gregor Leander, editors, *EUROCRYPT 2024, Part V*, volume 14655 of *LNCS*, pages 370–400. Springer, Cham, May 2024. (Cited on Page 4.)
- [HJ12] Dennis Hofheinz and Tibor Jager. Tightly secure signatures and public-key encryption. In Reihaneh Safavi-Naini and Ran Canetti, editors, *CRYPTO 2012*, volume 7417 of *LNCS*, pages 590–607. Springer, Berlin, Heidelberg, August 2012. (Cited on Page 5.)
- [HJK12] Dennis Hofheinz, Tibor Jager, and Edward Knapp. Waters signatures with optimal security reduction. In Marc Fischlin, Johannes Buchmann, and Mark Manulis, editors, *PKC 2012*, volume 7293 of *LNCS*, pages 66–83. Springer, Berlin, Heidelberg, May 2012. (Cited on Page 5.)
- [HJP18] Dennis Hofheinz, Dingding Jia, and Jiaxin Pan. Identity-based encryption tightly secure under chosen-ciphertext attacks. In Thomas Peyrin and Steven Galbraith, editors, *ASIACRYPT 2018, Part II*, volume 11273 of *LNCS*, pages 190–220. Springer, Cham, December 2018. (Cited on Page 5.)
- [HK22] Lucjan Hanzlik and Kamil Kluczniak. Explainable arguments. In Ittay Eyal and Juan A. Garay, editors, *FC 2022*, volume 13411 of *LNCS*, pages 59–79. Springer, Cham, May 2022. (Cited on Page 10, 14, 34.)
- [HKK⁺24] Goichiro Hanaoka, Shuichi Katsumata, Kei Kimura, Kaoru Takemure, and Shota Yamada. Tighter adaptive IBEs and VRFs: Revisiting waters’ artificial abort. *Cryptology ePrint Archive*, Paper 2024/1481, 2024. (Cited on Page 5.)
- [HKS15] Dennis Hofheinz, Jessica Koch, and Christoph Striecks. Identity-based encryption with (almost) tight security in the multi-instance, multi-ciphertext setting. In Jonathan Katz, editor, *PKC 2015*, volume 9020 of *LNCS*, pages 799–822. Springer, Berlin, Heidelberg, March / April 2015. (Cited on Page 5.)
- [IN83] Kazuharu Itakura and Katsuhiro Nakamura. A public-key cryptosystem suitable for digital multisignatures. *NEC Research & Development*, (71):1–8, 1983. (Cited on Page 5.)
- [JL00] Stanislaw Jarecki and Anna Lysyanskaya. Adaptively secure threshold cryptography: Introducing concurrency, removing erasures. In Bart Preneel, editor, *EUROCRYPT 2000*, volume 1807 of *LNCS*, pages 221–242. Springer, Berlin, Heidelberg, May 2000. (Cited on Page 5.)
- [KG20] Chelsea Komlo and Ian Goldberg. FROST: Flexible round-optimized Schnorr threshold signatures. In Orr Dunkelman, Michael J. Jacobson Jr., and Colin O’Flynn, editors, *SAC 2020*, volume 12804 of *LNCS*, pages 34–65. Springer, Cham, October 2020. (Cited on Page 3, 4, 25.)
- [KK12] Saqib A. Kakvi and Eike Kiltz. Optimal security proofs for full domain hash, revisited. In David Pointcheval and Thomas Johansson, editors, *EUROCRYPT 2012*, volume 7237 of *LNCS*, pages 537–553. Springer, Berlin, Heidelberg, April 2012. (Cited on Page 5.)
- [KMP16] Eike Kiltz, Daniel Masny, and Jiaxin Pan. Optimal security proofs for signatures from identification schemes. *Cryptology ePrint Archive*, Report 2016/191, 2016. (Cited on Page 17.)

- [KRB⁺24] Ioanna Karantaidou, Omar Renawi, Foteini Baldimtsi, Nikolaos Kamarinakis, Jonathan Katz, and Julian Loss. Blind multisignatures for anonymous tokens with decentralized issuance. Cryptology ePrint Archive, Paper 2024/1406, 2024. (Cited on Page 5.)
- [KRT24] Shuichi Katsumata, Michael Reichle, and Kaoru Takemure. Adaptively secure 5 round threshold signatures from MLWE/MSIS and DL with rewinding. In Leonid Reyzin and Douglas Stebila, editors, *CRYPTO 2024, Part VII*, volume 14926 of *LNCS*, pages 459–491. Springer, Cham, August 2024. (Cited on Page 3, 4, 5, 17, 25.)
- [KW03] Jonathan Katz and Nan Wang. Efficiency improvements for signature schemes with tight security reductions. In Sushil Jajodia, Vijayalakshmi Atluri, and Trent Jaeger, editors, *ACM CCS 2003*, pages 155–164. ACM Press, October 2003. (Cited on Page 5, 7.)
- [KYY18] Shuichi Katsumata, Shota Yamada, and Takashi Yamakawa. Tighter security proofs for GPV-IBE in the quantum random oracle model. In Thomas Peyrin and Steven Galbraith, editors, *ASIACRYPT 2018, Part II*, volume 11273 of *LNCS*, pages 253–282. Springer, Cham, December 2018. (Cited on Page 5.)
- [Lin24] Yehuda Lindell. Simple three-round multiparty schnorr signing with full simulatability. *CiC*, 1(1):25, 2024. (Cited on Page 3, 4, 25.)
- [LJY14] Benoît Libert, Marc Joye, and Moti Yung. Born and raised distributively: fully distributed non-interactive adaptively-secure threshold signatures with short shares. In Magnús M. Halldórsson and Shlomi Dolev, editors, *33rd ACM PODC*, pages 303–312. ACM, July 2014. (Cited on Page 5.)
- [LOS⁺06] Steve Lu, Rafail Ostrovsky, Amit Sahai, Hovav Shacham, and Brent Waters. Sequential aggregate signatures and multisignatures without random oracles. In Serge Vaudenay, editor, *EUROCRYPT 2006*, volume 4004 of *LNCS*, pages 465–485. Springer, Berlin, Heidelberg, May / June 2006. (Cited on Page 5.)
- [LP01] Anna Lysyanskaya and Chris Peikert. Adaptive security in the threshold setting: From cryptosystems to signature schemes. In Colin Boyd, editor, *ASIACRYPT 2001*, volume 2248 of *LNCS*, pages 331–350. Springer, Berlin, Heidelberg, December 2001. (Cited on Page 5.)
- [Mak22] Nikolaos Makriyannis. On the classic protocol for MPC schnorr signatures. Cryptology ePrint Archive, Paper 2022/1332, 2022. (Cited on Page 4, 5, 12, 25.)
- [MOR01] Silvio Micali, Kazuo Ohta, and Leonid Reyzin. Accountable-subgroup multisignatures: Extended abstract. In Michael K. Reiter and Pierangela Samarati, editors, *ACM CCS 2001*, pages 245–254. ACM Press, November 2001. (Cited on Page 5.)
- [MPSW19] Gregory Maxwell, Andrew Poelstra, Yannick Seurin, and Pieter Wuille. Simple schnorr multi-signatures with applications to bitcoin. *Des. Codes Cryptography*, 87(9):2139–2164, September 2019. (Cited on Page 5.)
- [NRS21] Jonas Nick, Tim Ruffing, and Yannick Seurin. MuSig2: Simple two-round Schnorr multisignatures. In Tal Malkin and Chris Peikert, editors, *CRYPTO 2021, Part I*, volume 12825 of *LNCS*, pages 189–221, Virtual Event, August 2021. Springer, Cham. (Cited on Page 5, 8, 40.)
- [NRSW20] Jonas Nick, Tim Ruffing, Yannick Seurin, and Pieter Wuille. MuSig-DN: Schnorr multisignatures with verifiably deterministic nonces. In Jay Ligatti, Xinming Ou, Jonathan Katz, and Giovanni Vigna, editors, *ACM CCS 2020*, pages 1717–1731. ACM Press, November 2020. (Cited on Page 5.)
- [Ped91] Torben P. Pedersen. A threshold cryptosystem without a trusted party (extended abstract) (rump session). In Donald W. Davies, editor, *EUROCRYPT’91*, volume 547 of *LNCS*, pages 522–526. Springer, Berlin, Heidelberg, April 1991. (Cited on Page 3.)

- [PW21] Jiaxin Pan and Benedikt Wagner. Short identity-based signatures with tight security from lattices. In Jung Hee Cheon and Jean-Pierre Tillich, editors, *Post-Quantum Cryptography*, pages 360–379, Cham, 2021. Springer International Publishing. (Cited on Page 5.)
- [PW22] Jiaxin Pan and Benedikt Wagner. Lattice-based signatures with tight adaptive corruptions and more. In Goichiro Hanaoka, Junji Shikata, and Yohei Watanabe, editors, *PKC 2022, Part II*, volume 13178 of *LNCS*, pages 347–378. Springer, Cham, March 2022. (Cited on Page 5.)
- [PW23] Jiaxin Pan and Benedikt Wagner. Chopsticks: Fork-free two-round multi-signatures from non-interactive assumptions. In Carmit Hazay and Martijn Stam, editors, *EUROCRYPT 2023, Part V*, volume 14008 of *LNCS*, pages 597–627. Springer, Cham, April 2023. (Cited on Page 5, 7, 8.)
- [PW24] Jiaxin Pan and Benedikt Wagner. Toothpicks: More efficient fork-free two-round multi-signatures. In Marc Joye and Gregor Leander, editors, *EUROCRYPT 2024, Part I*, volume 14651 of *LNCS*, pages 460–489. Springer, Cham, May 2024. (Cited on Page 5, 7, 8.)
- [QLH12] Haifeng Qian, Xiangxue Li, and Xinli Huang. Tightly secure non-interactive multisignatures in the plain public key model. *Informatica (Vilnius)*, 3, 01 2012. (Cited on Page 5.)
- [RRJ⁺22] Tim Ruffing, Viktoria Ronge, Elliott Jin, Jonas Schneider-Bensch, and Dominique Schröder. ROAST: Robust asynchronous schnorr threshold signatures. In Heng Yin, Angelos Stavrou, Cas Cremers, and Elaine Shi, editors, *ACM CCS 2022*, pages 2551–2564. ACM Press, November 2022. (Cited on Page 4, 25.)
- [RY07] Thomas Ristenpart and Scott Yilek. The power of proofs-of-possession: Securing multiparty signatures against rogue-key attacks. In Moni Naor, editor, *EUROCRYPT 2007*, volume 4515 of *LNCS*, pages 228–245. Springer, Berlin, Heidelberg, May 2007. (Cited on Page 5.)
- [Seu12] Yannick Seurin. On the exact security of Schnorr-type signatures in the random oracle model. In David Pointcheval and Thomas Johansson, editors, *EUROCRYPT 2012*, volume 7237 of *LNCS*, pages 554–571. Springer, Berlin, Heidelberg, April 2012. (Cited on Page 4, 5.)
- [Sho97] Victor Shoup. Lower bounds for discrete logarithms and related problems. In Walter Fumy, editor, *EUROCRYPT’97*, volume 1233 of *LNCS*, pages 256–266. Springer, Berlin, Heidelberg, May 1997. (Cited on Page 40.)
- [SS01] Douglas R. Stinson and Reto Stöbl. Provably secure distributed Schnorr signatures and a (t, n) threshold scheme for implicit certificates. In Vijay Varadharajan and Yi Mu, editors, *ACISP 01*, volume 2119 of *LNCS*, pages 417–434. Springer, Berlin, Heidelberg, July 2001. (Cited on Page 4.)
- [TZ22] Stefano Tessaro and Chenzhi Zhu. Short pairing-free blind signatures with exponential security. In Orr Dunkelman and Stefan Dziembowski, editors, *EUROCRYPT 2022, Part II*, volume 13276 of *LNCS*, pages 782–811. Springer, Cham, May / June 2022. (Cited on Page 3.)
- [TZ23] Stefano Tessaro and Chenzhi Zhu. Threshold and multi-signature schemes from linear hash functions. In Carmit Hazay and Martijn Stam, editors, *EUROCRYPT 2023, Part V*, volume 14008 of *LNCS*, pages 628–658. Springer, Cham, April 2023. (Cited on Page 3, 4, 5, 25.)
- [WQL09] Zecheng Wang, Haifeng Qian, and Zhibin Li. Adaptively secure threshold signature scheme in the standard model. *Informatica*, 20(4):591–612, dec 2009. (Cited on Page 5.)

Appendix

A More Details on Non-Interactive Argument Systems

In this section, we give formal definitions and proofs for non-interactive argument systems.

A.1 Formal Definitions

Here, we define non-interactive argument systems for **NP** relations. We focus on argument systems in the random oracle model. For all definitions related to argument systems, we do not explicitly define an advantage of the form $\text{Adv}(\lambda)$, but rather use ε_{zzk} to convey that for our instantiation this will hold for *any potentially unbounded* adversary, assuming only *polynomially many* random oracle queries. Before we can give the definition of an argument systems, we first formally define what we mean by an **NP** relation.

Definition 7 (NP Relation). Let $\mathcal{R}$ be a relation containing pairs $(\text{stmt}, \text{witn})$, which is implicitly parameterized by the security parameter λ . We say that $\mathcal{R}$ is an **NP** relation and call stmt the statement and witn the witness, if the following hold:

- *Size Constraints.* There are polynomials poly_1 and poly_2 such that for all $(\text{stmt}, \text{witn}) \in \mathcal{R}$, we have $|\text{stmt}| \leq \text{poly}_1(\lambda)$ and $|\text{witn}| \leq \text{poly}_2(|\text{stmt}|)$.
- *Efficiently Decidable.* The relation can be decided deterministically in polynomial time, i.e., there is a deterministic polynomial time algorithm that takes as input a pair $(\text{stmt}, \text{witn})$ and outputs 1 if and only if $(\text{stmt}, \text{witn}) \in \mathcal{R}$.

For such a relation, we define the induced language $\mathcal{L}_{\mathcal{R}} \subseteq \{0, 1\}^*$ by

$$\mathcal{L}_{\mathcal{R}} := \{\text{stmt} \in \{0, 1\}^* \mid \exists \text{witn} \in \{0, 1\}^* : (\text{stmt}, \text{witn}) \in \mathcal{R}\}.$$

Definition 8 (Non-Interactive Argument System). Let $\mathcal{R}$ be an **NP** relation and H be a random oracle. A non-interactive argument system for $\mathcal{R}$ with respect to $H: \mathcal{X} \rightarrow \mathcal{Y}$ is defined to be a pair $\text{AS} = (\text{Prove}, \text{VerProof})$ of PPT algorithms with oracle access to H and the following syntax:

- $\text{Prove}^H(\text{stmt}, \text{witn}) \rightarrow \pi$ takes as input a statement stmt and a witness witn , and outputs a proof π . We assume that the random coins are uniform over $\{0, 1\}^{\ell(\lambda)}$, i.e., we can write $\pi := \text{Prove}^H(\text{stmt}, \text{witn}; \rho)$ for $\rho \xleftarrow{\$} \{0, 1\}^{\ell(\lambda)}$.
- $\text{VerProof}^H(\text{stmt}, \pi) \rightarrow b$ is deterministic, takes as input a statement stmt and a proof π , and outputs a decision bit $b \in \{0, 1\}$.

Further, we require that the argument system is complete in the following sense: For any pair $(\text{stmt}, \text{witn}) \in \mathcal{R}$, we have

$$\Pr \left[\text{VerProof}^H(\text{stmt}, \pi) = 1 \mid \pi \leftarrow \text{Prove}^H(\text{stmt}, \text{witn}) \right] = 1,$$

where the probability is taken over the random coins of Prove and the random oracle H , which may be queried by external algorithms in an arbitrary way.

We need non-interactive arguments that are zero-knowledge, i.e., there is an efficient simulator that can generate proofs without access to the witness, which is typically accomplished by programming the random oracle. Note, however, that standard zero-knowledge is not enough in the context of adaptive corruptions: say the reduction simulates a proof on behalf of some honest party. When the adversary later corrupts this honest party, it would expect to learn the witness and well-distributed random coins that have been used to generate the proof from the witness. Therefore, we need a stronger form of zero-knowledge, in which the simulator can create appropriate random coins once it learns the witness. A similar notion called *explainable arguments* has been introduced by Hanzlik and Klucznik [HK22]. We call our definition explainable zero-knowledge. In our definition, the adversary runs in one of two games, and has to say in which game it runs. In the first game, it gets access to honestly generated proofs, for which it can later ask the game to reveal the random coins that have been used. In the second game, proofs are provided by the simulator without access to the witness. When the adversary wants to learn

Game $\mathbf{X-ZK}_{AS,Sim,b}^A(\lambda)$ 01 $b' \leftarrow \mathcal{A}^{\text{GetProof}_b, \text{GetCoins}_b, H}(1^\lambda)$ 02 return b' Oracle $\text{GetProof}_0(pid, stmt, witn)$ 03 if $St[pid] \neq \perp$: return $\perp$ 04 if $(stmt, witn) \notin \mathcal{R}$: return $\perp$ 05 $\rho \xleftarrow{\$} \{0, 1\}^{\ell(\lambda)}$, $St[pid] := \rho$ 06 $\pi := \text{Prove}^H(stmt, witn; \rho)$ 07 return π Oracle $\text{GetCoins}_0(pid)$ 08 $\rho := St[pid]$ 09 return ρ	Oracle $H(x)$ 10 if $h[x] = \perp$: $h \xleftarrow{\$} \mathcal{Y}$ 11 return $h[x]$ Oracle $\text{GetProof}_1(pid, stmt, witn)$ 12 if $St[pid] \neq \perp$: return $\perp$ 13 if $(stmt, witn) \notin \mathcal{R}$: return $\perp$ 14 $(\pi, St, h'[\cdot]) \leftarrow \text{Sim}(0, h[\cdot], stmt)$ 15 $St[pid] := (St, witn)$, $h[\cdot] := h'[\cdot]$ 16 return π Oracle $\text{GetCoins}_1(pid)$ 17 $(St, witn) := St[pid]$ 18 return $\rho := \text{Sim}(1, h[\cdot], St, witn)$
--	--

Figure 5: The games $\mathbf{X-ZK}$ modeling explainable zero-knowledge for a non-interactive argument system $AS = (\text{Prove}, \text{VerProof})$ for an $\mathbf{NP}$ relation $\mathcal{R}$ with respect to a random oracle $H: \mathcal{X} \rightarrow \mathcal{Y}$ and an adversary $\mathcal{A}$. Note that the simulator Sim can update the random oracle in oracle GetProof_1 by outputting a new map h' .

random coins, the witness is given to the simulator, and the simulator is expected to generate the random coins. To formalize that the simulator can program the random oracle, we allow the simulator to output a new map that lazily implements the random oracle.

Definition 9 (Explainable Zero-Knowledge). Let $\mathcal{R}$ be an $\mathbf{NP}$ relation and $H: \mathcal{X} \rightarrow \mathcal{Y}$ be a random oracle. Let AS be a non-interactive argument system for $\mathcal{R}$. We say that AS satisfies ε_{zk} -explainable zero-knowledge, if there is a PPT algorithm Sim (called the simulator) such that for any algorithm $\mathcal{A}$, we have

$$\left| \Pr \left[\mathbf{X-ZK}_{AS,Sim,0}^A(\lambda) \Rightarrow 1 \right] - \Pr \left[\mathbf{X-ZK}_{AS,Sim,1}^A(\lambda) \Rightarrow 1 \right] \right| \leq \varepsilon_{\text{zk}},$$

where the games $\mathbf{X-ZK}_{AS,Sim,b}^A(\lambda)$ are defined in Figure 5. The term ε_{zk} is allowed to depend on the number of random oracle queries $\mathcal{A}$ makes.

Next, we define a weak form of simulation-soundness in presence of explained proofs. That is, in our simulation-soundness game, the adversary gets access to simulated proofs and random coins that are output by the simulator, similar to the second game of explainable zero-knowledge. Then, the goal of the adversary is to output a proof that verifies for a wrong statement. In our definition, the adversary does not get access to simulated proofs for invalid statements.

Game $\mathbf{W-SIM-SND}_{AS,Sim}^A(\lambda)$ 01 $\mathcal{A}^{\text{GetProof}_1, \text{GetCoins}_1, H, \text{ForgeProof}}(1^\lambda)$ 02 return win	Oracle $\text{ForgeProof}(stmt, \pi)$ 03 if $stmt \in \mathcal{L}_{\mathcal{R}}$: return 04 if $\text{VerProof}^H(stmt, \pi) = 1$: win := 1
---	--

Figure 6: The game $\mathbf{W-SIM-SND}$ modeling weak simulation-soundness for a non-interactive argument system $AS = (\text{Prove}, \text{VerProof})$ for an $\mathbf{NP}$ relation $\mathcal{R}$ with respect to a random oracle $H: \mathcal{X} \rightarrow \mathcal{Y}$ and an adversary $\mathcal{A}$. Oracles $\text{GetProof}_1, \text{GetCoins}_1, H$ are as in Figure 5.

Definition 10 (Weak Simulation-Soundness). Let $\mathcal{R}$ be an $\mathbf{NP}$ relation and $H: \mathcal{X} \rightarrow \mathcal{Y}$ be a random oracle. Let AS be a non-interactive argument system for $\mathcal{R}$ and Sim be a PPT algorithm. Then, we say that AS satisfies $\varepsilon_{\text{wssnd}}$ -weak simulation-soundness with respect to Sim , if for any algorithm $\mathcal{A}$, we have

$$\Pr \left[\mathbf{W-SIM-SND}_{AS,Sim}^A(\lambda) \Rightarrow 1 \right] \leq \varepsilon_{\text{wssnd}},$$

where the game $\mathbf{W-SIM-SND}_{AS,Sim}^A(\lambda)$ is defined in Figure 6. The term $\varepsilon_{\text{wssnd}}$ is allowed to depend on the number of random oracle queries $\mathcal{A}$ makes.

As said before, we do not give the adversary access to simulated proofs for invalid statements. It is well-known that in this case, simulation-soundness follows from standard soundness, as defined next, and zero-knowledge. We show this for completeness in Lemma 5.

Definition 11 (Soundness). Let $\mathcal{R}$ be an **NP** relation and $H: \mathcal{X} \rightarrow \mathcal{Y}$ be a random oracle. Let $\mathcal{AS}$ be a non-interactive argument system for **NP**. Then, we say that $\mathcal{AS}$ satisfies ε_{snd} -soundness, if for any algorithm $\mathcal{A}$, we have

$$\Pr \left[\text{VerProof}^H(\text{stmt}, \pi) = 1 \wedge \text{stmt} \notin \mathcal{L}_{\mathcal{R}} \mid (\text{stmt}, \pi) \leftarrow \mathcal{A}^H(1^\lambda) \right] \leq \varepsilon_{\text{snd}}.$$

The term ε_{snd} is allowed to depend on the number of random oracle queries $\mathcal{A}$ makes.

Lemma 5. Let $\mathcal{R}$ be an **NP** relation and $H: \mathcal{X} \rightarrow \mathcal{Y}$ be a random oracle. Let $\mathcal{AS}$ be a non-interactive argument system for $\mathcal{R}$ and Sim be a PPT algorithm. Assume that $\mathcal{AS}$ satisfies ε_{snd} -soundness and that it satisfies ε_{xzk} -explainable zero-knowledge with simulator Sim . Then, it satisfies $\varepsilon_{\text{wssnd}}$ -weak simulation-soundness with respect to Sim , where

$$\varepsilon_{\text{wssnd}} \leq \varepsilon_{\text{xzk}} + \varepsilon_{\text{snd}}.$$

Proof. Let $\mathcal{A}$ be any algorithm. Our goal is to show that

$$\Pr \left[\mathbf{W}\text{-SIM-SND}_{\mathcal{AS}, \text{Sim}}^{\mathcal{A}}(\lambda) \Rightarrow 1 \right] \leq \varepsilon_{\text{xzk}} + \varepsilon_{\text{snd}}.$$

Recall that in $\mathbf{W}\text{-SIM-SND}_{\mathcal{AS}, \text{Sim}}^{\mathcal{A}}(\lambda)$, the adversary $\mathcal{A}$ gets access to oracles $\text{GetProof}_1, \text{GetCoins}_1, H, \text{ForgeProof}$, and the game outputs the variable win (see Figure 6). Now, consider a game $\mathbf{G}$, which is exactly as experiment $\mathbf{W}\text{-SIM-SND}_{\mathcal{AS}, \text{Sim}}^{\mathcal{A}}(\lambda)$, but the oracles $\text{GetProof}_1, \text{GetCoins}_1$ are replaced by the oracles $\text{GetProof}_0, \text{GetCoins}_0$ given in Figure 5. We claim that it follows from ε_{xzk} -explainable zero-knowledge that

$$\left| \Pr \left[\mathbf{W}\text{-SIM-SND}_{\mathcal{AS}, \text{Sim}}^{\mathcal{A}}(\lambda) \Rightarrow 1 \right] - \Pr [\mathbf{G} \Rightarrow 1] \right| \leq \varepsilon_{\text{xzk}}.$$

To see why this holds, consider the following (potentially inefficient reduction), running in $\mathbf{X}\text{-ZK}_{\mathcal{AS}, \text{Sim}, b}^{\mathcal{A}}(\lambda)$ for some $b \in \{0, 1\}$:

- The reduction gets access to oracles $\text{GetProof}_b, \text{GetCoins}_b, H$.
- The reduction runs $\mathcal{A}$ on oracles $\text{GetProof}_b, \text{GetCoins}_b, H, \text{ForgeProof}$. It simulates ForgeProof and sets win accordingly via brute-force, but without making any additional queries to H .
- Finally, the reduction outputs win .

Clearly, if $b = 0$, the reduction perfectly simulates $\mathbf{G}$ for $\mathcal{A}$, whereas if $b = 1$, it perfectly simulates $\mathbf{W}\text{-SIM-SND}_{\mathcal{AS}, \text{Sim}}^{\mathcal{A}}(\lambda)$. As the number of random oracle queries does not increase, the claim follows. Hence, it remains to bound the probability that $\mathbf{G}$ outputs 1. This can easily be done via an (again, inefficient) reduction breaking soundness. This reduction is as follows:

- The reduction runs in the soundness game, i.e., it has access to a random oracle H .
- The reduction simulates game $\mathbf{G}$ honestly for $\mathcal{A}$. Note that in $\mathbf{G}$, the random oracle is never programmed.
- Simulation of oracle ForgeProof is potentially inefficient but does not increase the number of random oracle queries.
- When $\mathcal{A}$ makes the first query $\text{ForgeProof}(\text{stmt}, \pi)$ for which $\text{win} = 1$ is set, the reduction terminates with output (stmt, π) .

As the reduction makes as many random oracle queries as $\mathcal{A}$ and breaks soundness whenever $\mathbf{G}$ outputs 1, we get

$$\Pr [\mathbf{G} \Rightarrow 1] \leq \varepsilon_{\text{snd}}.$$

The lemma follows. $\square$

A.2 Proofs for Our Construction

Here, we prove Lemmata 1 and 2. Before we do that, we show a generic statistical lemma that will turn out to be useful. Intuitively, it states that the aggregation step of our argument system is sound. We also recall a useful statistical lemma taken from [BLT⁺24].

Lemma 6. *Let $\text{TLF} = (\text{Gen}, \text{T})$ be a tagged linear function family. For every fixed parameters par and tags $g, h \in \mathcal{T}$, define the set*

$$\text{Im}(\text{par}, g, h) := \{(X_1, X_2) \in \mathcal{R}^2 \mid \exists x \in \mathcal{D} : \text{T}(g, x) = X_1 \wedge \text{T}(h, x) = X_2\}.$$

Then, for any (even unbounded) algorithm $\mathcal{A}$, we have

$$\Pr \left[(X \notin \mathcal{I} \vee R \notin \mathcal{I}) \wedge \bar{X} \in \mathcal{I} \mid \begin{array}{l} \text{par} \leftarrow \text{Gen}(1^\lambda), \\ (g, h, R, X) \leftarrow \mathcal{A}(\text{par}), \\ \gamma \xleftarrow{\$} \mathcal{S}, \bar{X} := R + \gamma X \end{array} \right] \leq \frac{1}{|\mathcal{S}|},$$

where $\mathcal{I} := \text{Im}(\text{par}, g, h)$ and $R, X, \bar{X} \in \mathcal{R}^2$.

Proof. Consider the experiment in the statement, and define $\mathcal{I} := \text{Im}(\text{par}, g, h) \subseteq \mathcal{R}^2$. Then, it is easy to see that $\mathcal{I}$ is a subspace of the $\mathcal{S}$ -vector space $\mathcal{R}^2$. Hence, there is a subspace $\mathcal{U} \subseteq \mathcal{R}^2$ such that $\mathcal{U} \oplus \mathcal{I} = \mathcal{R}^2$, i.e., any $W \in \mathcal{R}^2$ can be written as $W = U_W + I_W$ for some uniquely defined $U_W \in \mathcal{U}$ and $I_W \in \mathcal{I}$. Now, let $X, R, \bar{X}$ be as in the experiment. Then, write $X = U_X + I_X$ and $R = U_R + I_R$ with $U_X, U_R \in \mathcal{U}$ and $V_X, V_R \in \mathcal{I}$. With that, note that

$$\bar{X} = R + \gamma X = \underbrace{(U_R + \gamma U_X)}_{\in \mathcal{U}} + \underbrace{(I_R + \gamma I_X)}_{\in \mathcal{I}}.$$

In particular, the probability that we want to bound becomes

$$\Pr [\bar{X} \in \mathcal{I} \mid R \notin \mathcal{I} \vee X \notin \mathcal{I}] \leq \Pr_{\gamma} [U_R + \gamma U_X = 0 \mid U_R \neq 0 \vee U_X \neq 0] \leq \frac{1}{|\mathcal{S}|}.$$

□

Lemma 7 ([BLT⁺24]). *Let $\text{TLF} = (\text{Gen}, \text{T})$ be a tagged linear function family. For every fixed parameters par and tags $g, h \in \mathcal{T}$, define the set*

$$\text{Im}(\text{par}, g, h) := \{(X_1, X_2) \in \mathcal{R}^2 \mid \exists x \in \mathcal{D} : \text{T}(g, x) = X_1 \wedge \text{T}(h, x) = X_2\}.$$

Then, for any (even unbounded) algorithm $\mathcal{A}$, we have

$$\Pr \left[\begin{array}{l} (X_1, X_2) \notin \text{Im}(\text{par}, g, h) \\ \wedge \text{T}(g, s) = c \cdot X_1 + R_1 \\ \wedge \text{T}(h, s) = c \cdot X_2 + R_2 \end{array} \mid \begin{array}{l} \text{par} \leftarrow \text{Gen}(1^\lambda), \\ (St, g, h, X_1, X_2, R_1, R_2) \leftarrow \mathcal{A}(\text{par}), \\ c \xleftarrow{\$} \mathcal{S}, s \leftarrow \mathcal{A}(St, c) \end{array} \right] \leq \frac{1}{|\mathcal{S}|}.$$

Proof of Lemma 1. We prove explainable zero-knowledge of $\text{AS}[\text{TLF}]$. To this end, we give an efficient simulator Sim , which has two jobs:

- *Simulating Proofs.* On input $(0, h[\cdot], \text{stmt})$, where $h[\cdot]$ is the map defining the current state of the random oracle $\text{H}: \{0, 1\}^* \rightarrow \mathcal{S}$ and stmt is a valid statement, the simulator needs to simulate a proof π for this statement. It is allowed to program the random oracle by outputting a new map $h'[\cdot]$ that represents the new state of the random oracle.
- *Simulating Random Coins.* At some later point in time, the simulator may get input $(1, h[\cdot], St, \text{wtn})$, where again $h[\cdot]$ represents the random oracle, St is a state the simulator can output when simulating a proof, and wtn is a witness for the statement for which it simulated the proof. The task of the simulator is to output well-distributed random coins ρ such that π could have been generated using the witness and these coins. In the case of $\text{AS}[\text{TLF}]$, the random coins are simply $w \xleftarrow{\$} \mathcal{D}$.

We now describe how the simulator works for our scheme $\text{AS}[\text{TLF}]$:

- $\text{Sim}(0, h[\cdot], \text{stmt}) \rightarrow (\pi, St, h'[\cdot])$:
 1. Parse the statement as $(h, R_1, R_2, X_1, X_2) := \text{stmt}$.
 2. Set $\gamma := H(0, \text{stmt})$, which potentially updates map $h[\cdot]$.
 3. Define $\bar{X} := (R_1 + \gamma X_1, R_2 + \gamma X_2)$ and $\overline{\text{stmt}} := (h, \bar{X})$.
 4. Sample $c \xleftarrow{\$} \mathcal{S}$ and $z \xleftarrow{\$} \mathcal{D}$ and set $W := \varphi_{g,h}(z) - c\bar{X}$.
 5. If $h[1, \overline{\text{stmt}}, W] \neq \perp$, abort. Otherwise, set $h[1, \overline{\text{stmt}}, W] := c$.
 6. Set $\pi := (c, z)$ and $St := (\gamma, c, z)$.
 7. Define $h'[\cdot]$ to be the map $h[\cdot]$ with all the changes so far.
 8. Return $(\pi, St, h'[\cdot])$.
- $\text{Sim}(1, h[\cdot], St, \text{witn}) \rightarrow \rho$:
 1. Parse $(\gamma, z, c) := St$ and $(r, x) := \text{witn}$.
 2. Set $\bar{x} := r + \gamma x$ and return $\rho := w := z - c\bar{x}$.

We now turn to the analysis of our simulator. First, it is clear that Sim is PPT. Second, we need to argue that for any algorithm $\mathcal{A}$, making at most Q_H queries to H and Q queries to the oracles GetProof and GetCoins and running either in the game with honestly generated proofs or with the simulator, the distinguishing advantage is bounded by ε_{zk} as in the lemma. To this end, we first bound the probability that the simulator will ever abort. Namely, by regularity of TLF and the assumption that (par, g) is in the regularity set, we know that the first component of $\varphi_{g,h}(z)$ for $z \xleftarrow{\$} \mathcal{D}$ is uniformly distributed over $\mathcal{R}$, and so is the first component of $W = \varphi_{g,h}(z) - c\bar{X}$. Therefore, for any fixed invocation of the simulator, the probability that it aborts because $h[1, \overline{\text{stmt}}, W]$ is already defined is at most $Q_H/|\mathcal{R}|$. With a union bound over the at most Q invocations of the simulator, the probability of an abort is therefore bounded by $QQ_H/|\mathcal{R}|$. Now, condition on no abort happening. In this case, note that the simulator performs the aggregation step (computing $\bar{X}$ from (R_1, R_2) and (X_1, X_2)) exactly as the honest prover algorithm would do. Therefore, it is sufficient to observe that for any $\bar{x} \in \mathcal{D}$, the following distributions are the same:

- *Real Distribution.* Output $(\bar{x}, w, c, z)$, where $w \xleftarrow{\$} \mathcal{D}$, $c \xleftarrow{\$} \mathcal{S}$, and $z := c\bar{x} + w$.
- *Simulated Distribution.* Output $(\bar{x}, w, c, z)$, where $z \xleftarrow{\$} \mathcal{D}$, $c \xleftarrow{\$} \mathcal{S}$, and $w := z - c\bar{x}$.

This is clear, concluding the proof. $\square$

Proof of Lemma 2. We prove soundness of $\text{AS}[\text{TLF}]$. To this end, consider the soundness experiment for an adversary $\mathcal{A}$. The adversary $\mathcal{A}$ is assumed to have access to the parameters par, g specifying the relation $\mathcal{R}_{\text{TLF}}[\text{par}, g]$. It gets access to the random oracle $H: \{0, 1\}^* \rightarrow \mathcal{S}$ and the security parameter 1^λ as input. The adversary outputs a statement $\text{stmt} = (h, R_1, R_2, X_1, X_2) \in \mathcal{T} \times \mathcal{R}^4$ and a proof $\pi = (c, z) \in \mathcal{S} \times \mathcal{D}$. The adversary wins the game if the statement is invalid, but the proof verifies, i.e., if the following hold:

1. We have $(X_1, X_2) \notin \text{Im}(\text{par}, g, h)$ or $(R_1, R_2) \notin \text{Im}(\text{par}, g, h)$, using the notation of Lemmata 6 and 7.
2. We have $H(1, \overline{\text{stmt}}, W) = c$ for $\gamma := H(0, \text{stmt})$, $\bar{X} := (R_1 + \gamma X_1, R_2 + \gamma X_2)$, $\overline{\text{stmt}} := (h, \bar{X})$, and $W := \varphi_{g,h}(z) - c\bar{X}$.

We now consider the following events **BatchBreak**:

- **Event BatchBreak:** This event occurs, if $\bar{X} \in \text{Im}(\text{par}, g, h)$.
- **Event Win:** This event occurs, if $\mathcal{A}$ wins, as specified above. Our goal is to bound the probability of this event.

We can use Lemma 6 and a union bound over all Q_H random oracle queries to get

$$\Pr[\text{BatchBreak}] \leq \frac{Q_H}{|\mathcal{S}|}.$$

Now, condition on BatchBreak not occurring. Then in particular, we have that $\bar{X} \notin \text{Im}(\text{par}, g, h)$. We can therefore easily build a reduction that runs in the experiment in Lemma 7: it would guess the random oracle query $H(1, \overline{\text{stmt}}, W)$ associated with the adversaries output. When this query occurs, it would output its state and $g, h, \bar{X}, W$ to the experiment in Lemma 7 and program the random oracle to the challenge that it receives back. Finally, when $\mathcal{A}$ terminates and the guess was correct (which happens with probability $1/Q_H$), it would output z to the experiment. By Lemma 7, we have

$$\Pr[\text{Win}] \leq \Pr[\text{BatchBreak}] + \Pr[\text{Win} \mid \neg \text{BatchBreak}] \leq \frac{Q_H}{|\mathcal{S}|} + \frac{Q_H}{|\mathcal{S}|} = \frac{2Q_H}{|\mathcal{S}|}.$$

□

B More Details on Our Instantiations

We define the AOMCDH and DDH assumptions, which are used in our instantiations of tagged linear functions.

Definition 12 (AOMCDH Assumption). Let GGen be a group generation algorithm. That is, on input 1^λ , it outputs the description of a prime order group $\mathbb{G}$. The description contains the prime order p and a generator g of $\mathbb{G}$, and a description of the group operation. Consider the game **AOMCDH** in Figure 7. We say that the t -AOMCDH assumption holds relative to GGen , if for all PPT algorithms $\mathcal{A}$, the following advantage is negligible:

$$\text{Adv}_{\mathcal{A}, \text{GGen}}^{t\text{-AOMCDH}}(\lambda) := \Pr \left[t\text{-AOMCDH}_{\text{GGen}}^{\mathcal{A}}(\lambda) \Rightarrow 1 \right].$$

Game $t\text{-AOMCDH}_{\text{GGen}}^{\mathcal{A}}(\lambda)$	Oracle $\text{Inv}(\alpha_0, \dots, \alpha_t)$
01 $(\mathbb{G}, g, p) \leftarrow \text{GGen}(1^\lambda)$	07 if $q \geq t$: return $\perp$
02 $h \xleftarrow{\$} \mathbb{G}, x_0, \dots, x_t \xleftarrow{\$} \mathbb{Z}_p$	08 $q := q + 1$
03 for $i \in [t]$: $X_i := g^{x_i}$	09 $x := \sum_{i=0}^t \alpha_i x_i$
04 $(X'_i)_{i=0}^t \leftarrow \mathcal{A}^{\text{Inv}}(\mathbb{G}, g, p, h, (X_i)_{i=0}^t)$	10 return x
05 if $\forall i \in [t] X'_i = h^{x_i}$: return 1	
06 return 0	

Figure 7: The game **AOMCDH** from the definition of the AOMCDH assumption for an adversary $\mathcal{A}$.

The authors [BLT⁺24] show that this assumption is implied by the algebraic one-more discrete logarithm (AOMDL) assumption [NRS21] in the algebraic group model. Further, Bauer et al. [BFP21] give a bound for AOMDL in the generic group model (GGM) [Sho97]. As a direct implication, the advantage of any adversary against AOMCDH is bounded by the same probability, namely $\Theta(t^2/(p - t^2) + 1/p)$.

Definition 13 (DDH Assumption). Let GGen be a group generation algorithm. That is, on input 1^λ , it outputs the description of a prime order group $\mathbb{G}$. The description contains the prime order p and a generator g of $\mathbb{G}$, and a description of the group operation. We say that the DDH assumption holds relative to GGen , if for all PPT algorithms $\mathcal{A}$, the following advantage is negligible:

$$\begin{aligned} \text{Adv}_{\mathcal{A}, \text{GGen}}^{\text{DDH}}(\lambda) := & \left| \Pr \left[\mathcal{A}(\mathbb{G}, p, g, h, g^a, h^a) = 1 \mid \begin{array}{l} (\mathbb{G}, g, p) \leftarrow \text{GGen}(1^\lambda), \\ h \xleftarrow{\$} \mathbb{G}, a \xleftarrow{\$} \mathbb{Z}_p \end{array} \right] \right. \\ & \left. - \Pr \left[\mathcal{A}(\mathbb{G}, p, g, h, u, v) = 1 \mid \begin{array}{l} (\mathbb{G}, g, p) \leftarrow \text{GGen}(1^\lambda), \\ h, u, v \xleftarrow{\$} \mathbb{G} \end{array} \right] \right|. \end{aligned}$$

C Pseudocode

Alg Setup(1^λ) <pre> 11 $\text{par}' \leftarrow \text{TLF.Gen}(1^\lambda), g \xleftarrow{\\$} \mathcal{T}$ 12 return $\text{par} := (\text{par}', g)$ Alg Gen(par) 13 $a_0, \dots, a_t \xleftarrow{\\$} \mathcal{D}$ 14 for $i \in [n] : \text{sk}_i := \sum_{j=0}^t a_j i^j$ 15 $\text{pk} := \text{pk}_0 := \text{T}(g, a_0)$ 16 return $(\text{pk}, \text{sk}_1, \dots, \text{sk}_n)$ Alg Sig₀($S, i, \text{sk}_i, \text{m}$) 17 $\varrho_i \xleftarrow{\\$} \{0, 1\}^{2\lambda}, r_i \xleftarrow{\\$} \mathcal{D}$ 18 $R_i^{(1)} := \text{T}(g, r_i)$ 19 $\text{com}_i := \tilde{\text{H}}(S, i, R_i^{(1)})$ 20 $\text{pm}_1 := (\varrho_i, \text{com}_i)$ 21 $St_1 := (S, i, \text{sk}_i, \text{m}, r_i, \text{pm}_1)$ 22 return (pm_1, St_1) Alg Sig₁($St_1, \mathcal{M}_1$) 23 parse $(S, i, \text{sk}_i, \text{m}, r_i, \text{pm}_1) := St_1$ 24 parse $((\varrho_j, \text{com}_j))_{j \in S} := \mathcal{M}_1$ 25 if $(\varrho_i, \text{com}_i) \neq \text{pm}_1 : \text{return } \perp$ 26 $\varrho := \hat{\text{H}}(S, \text{m}, (\varrho_j)_{j \in S})$ 27 $h := \text{H}(\text{m}, \varrho)$ 28 $\text{pk}_i^{(2)} := \text{T}(h, \text{sk}_i)$ 29 $R_i^{(2)} := \text{T}(h, r_i)$ 30 $\text{stmt} := (h, R_i^{(1)}, R_i^{(2)}, \text{pk}_i, \text{pk}_i^{(2)})$ 31 $\text{witn} := (r_i, \text{sk}_i)$ 32 $\rho \xleftarrow{\\$} \mathcal{D}$ 33 $\pi_i := \text{Prove}(\text{stmt}, \text{witn}; \rho)$ 34 $\text{pm}_2 := (\text{pk}_i^{(2)}, R_i^{(2)}, R_i^{(1)}, \pi_i)$ 35 $St_2 := (\mathcal{M}_1, St_1, h, \varrho, \rho, \text{pm}_2)$ 36 return (pm_2, St_2) </pre>	Alg Sig₂($St_2, \mathcal{M}_2$) <pre> 37 parse $(\mathcal{M}_1, St_1, h, \varrho, \rho, \text{pm}_2) := St_2$ 38 parse $(S, i, \text{sk}_i, \text{m}, r_i, \text{pm}_1) := St_1$ 39 parse $((\varrho_j, \text{com}_j))_{j \in S} := \mathcal{M}_1$ 40 parse $((\text{pk}_j^{(2)}, R_j^{(2)}, R_j^{(1)}, \pi_j))_{j \in S} := \mathcal{M}_2$ 41 if $(\text{pk}_i^{(2)}, R_i^{(2)}, R_i^{(1)}, \pi_i) \neq \text{pm}_2 :$ 42 return $\perp$ 43 for $j \in S :$ 44 $\text{stmt}_j := (h, R_j^{(1)}, R_j^{(2)}, \text{pk}_j, \text{pk}_j^{(2)})$ 45 if $\exists j \in S$ s.t. $\tilde{\text{H}}(S, j, R_j^{(1)}) \neq \text{com}_j$ 46 $\vee \text{VerProof}(\text{stmt}_j, \pi_j) = 0 : \text{return } \perp$ 47 $R^{(1)} := \sum_{j \in S} R_j^{(1)}$ 48 $R^{(2)} := \sum_{j \in S} R_j^{(2)}$ 49 $\text{pk}^{(2)} := \sum_{j \in S} \ell_{j,S} \cdot \text{pk}_j^{(2)}$ 50 $c := \bar{\text{H}}(\text{pk}, \text{pk}^{(2)}, R^{(1)}, R^{(2)}, \text{m}, \varrho)$ 51 $s_i := c \cdot \ell_{i,S} \cdot \text{sk}_i + r_i$ 52 return $\text{pm}_3 := s_i$ Alg Combine($S, \text{m}, \mathcal{M}_1, \mathcal{M}_2, \mathcal{M}_3$) 53 parse $((\varrho_j, \text{com}_j))_{j \in S} := \mathcal{M}_1$ 54 parse $((\text{pk}_j^{(2)}, R_j^{(2)}, R_j^{(1)}, \pi_j))_{j \in S} := \mathcal{M}_2$ 55 parse $(s_j)_{j \in S} := \mathcal{M}_3$ 56 $R^{(1)} := \sum_{j \in S} R_j^{(1)}$ 57 $R^{(2)} := \sum_{j \in S} R_j^{(2)}$ 58 $\text{pk}^{(2)} := \sum_{j \in S} \ell_{j,S} \cdot \text{pk}_j^{(2)}$ 59 $\varrho := \hat{\text{H}}(S, \text{m}, (\varrho_j)_{j \in S})$ 60 $c := \bar{\text{H}}(\text{pk}, \text{pk}^{(2)}, R^{(1)}, R^{(2)}, \text{m}, \varrho)$ 61 $s := \sum_{j \in S} s_j$ 62 return $\sigma := (\text{pk}^{(2)}, c, s, \varrho)$ Alg Ver($\text{pk}, \text{m}, \sigma$) 63 parse $(\text{pk}^{(2)}, c, s, \varrho) := \sigma$ 64 $h := \text{H}(\text{m}, \varrho)$ 65 $R^{(1)} := \text{T}(g, s) - c \cdot \text{pk}$ 66 $R^{(2)} := \text{T}(h, s) - c \cdot \text{pk}^{(2)}$ 67 if $c = \bar{\text{H}}(\text{pk}, \text{pk}^{(2)}, R^{(1)}, R^{(2)}, \text{m}, \varrho) :$ 68 return 1 69 return 0 </pre>
---	---

Figure 8: The three-round (t, n) -threshold signature scheme $\text{Twinkle-T}[\text{TLF}] = (\text{Setup}, \text{Gen}, \text{Sig}, \text{Ver})$ for a tagged linear function family TLF . The scheme $\text{AS}[\text{TLF}] = (\text{Prove}, \text{VerProof})$ is as in Section 4.